Java Networking

#	Scenario (Short)	Topic / Concept	Level	Role Focus
1	Verify if tracking server (IP + port) is reachable	Network Connectivity / Socket Basics	L1	Backend Core
2	Design multiplayer game server with many players	Multi-Threaded Socket Server	L3	Backend Core
3	Check stock API availability before high-freq calls	API Health Check / HTTP	L2	Backend Core
4	Add AES encryption layer on top of HTTPS	Extra Security / Encryption	L4	Secure Backend
5	Auto-reconnect TCP trading client on disconnect	TCP Reconnect Logic	L3	Backend Core
6	WebSocket auto-reconnect for real-time scoreboard	WebSocket Reconnection	L3	Full Stack Backend
7	Follow HTTP redirects (3xx) for regional booking	HTTP Redirect Handling	L2	Backend Core
8	Use different HTTP proxies for different APIs	Proxy Configuration	L3	Backend Core
9	Debug TLS handshake failures & cert issues	SSL/TLS Diagnostics	L4	Security Backend
10	Push live cricket scores to connected clients	Streaming Server (Sockets)	L3	Backend Core
11	Serialize / deserialize JSON for API compatibility	Data Exchange / Serialization	L2	Backend Core
12	Monitor multiple URLs & log status with timeouts	HTTP Monitoring	L2	Backend Core
13	Show upload progress for large file backup	HTTP File Upload (Progress)	L3	Backend Core
14	Send compressed JSON (GZIP) to save bandwidth	GZIP Compression + HTTP	L3	Backend Core
15	Parallel video download via HTTP Range requests	Parallel Download Optimization	L3	Backend Core
16	Stream real-time stock prices via SSE	Server-Sent Events (SSE)	L3	Backend Core

17	Two-way chat room using WebSockets	Real-Time Communication	L3	Full Stack Backend
18	Download protected report using HTTP Basic Auth	Authentication Headers	L2	Backend Core
19	High-concurrency stock feed server with NIO	Java NIO (Non-Blocking IO)	L4	Advanced Backend
20	File transfer between two machines via TCP	Socket Streams & Protocol Design	L3	Backend Core
21	Stream huge JSON response without loading all	Streaming Parser / Memory Efficiency	L3	Backend Core
22	Configure Mutual TLS (client cert) for banking	Mutual TLS Security	L4	Secure Backend
23	Restrict admin server to trusted IP ranges	Access Control / Firewall Logic	L3	Security Backend
24	Add API keys in custom HTTP headers securely	Auth Header Injection	L2	Backend Core
25	Detect hostname → IP change (Dynamic DNS)	DNS Resolution Tracking	L3	Networking Core
26	Reuse HTTP connections with Keep-Alive	HTTP Connection Reuse	L2	Backend Core
27	Configure TCP connect & read timeouts	Socket Timeouts	L2	Backend Core
28	Enforce HTTPS and validate certificates	HTTPS Validation / TrustStore	L3	Security Backend
29	Send lightweight heartbeats over UDP	UDP Messaging / Heartbeat	L3	Backend Core
30	Verify Content-Type before saving downloads	HTTP Response Validation	L2	Backend Core
31	Use conditional GET (ETag/Last-Modified) for caching	HTTP Caching / Validation	L3	Backend Core
32	Communicate with custom TCP protocol (length-prefixed)	Custom Protocol Design	L4	Advanced Backend
33	Check server reachability via TCP probe	Socket Connectivity Test	L1	Backend Core

34	Build simple client-server time service	Socket Programming Basics	L1	Backend Core
35	Call weather API with API-key authentication	HTTP Client Usage	L1	Backend Core
36	Use Mutual TLS for enterprise connection	Security / TLS Auth	L4	Secure Backend
37	Conditional GET with validators for docs	HTTP Caching / Optimization	L3	Backend Core
38	Keep-alive across multiple requests	Persistent HTTP Connections	L2	Backend Core
39	Send JSON in HTTP POST body	JSON Request Handling	L1	Backend Core
40	Download large files in chunks (stream to disk)	HTTP Streaming Download	L2	Backend Core
41	Perform DNS lookup for domain (all records)	DNS Resolution	L2	Networking Core
42	Reverse DNS lookup (PTR)	Reverse Lookup / Networking	L2	Networking Core
43	Check if local port is available	Socket Port Binding Test	L2	Backend Core
44	Encode/decode URL params safely (UTF-8)	URL Encoding / Decoding	L1	Backend Core
45	Maintain cookies across HTTP requests	HTTP State Management	L2	Backend Core
46	Resume downloads with HTTP Range requests	Partial Download / Resuming	L2	Backend Core
47	Upload file with multipart/form-data	HTTP File Upload	L3	Backend Core
48	Get local machine hostname and IP	InetAddress Utilities	L1	Networking Core
49	Connect to test server with self-signed cert	SSLContext Configuration	L3	Security Backend
50	Throttle HTTP requests (client-side rate limit)	Rate Limiting / Concurrency	L3	Backend Core
51	Run many HTTP requests asynchronously	Async HTTP Client (Pool)	L3	Backend Core

52	Connect through authenticating HTTP proxy	Proxy Auth Configuration	L3	Backend Core
53	Broadcast chat messages on LAN (UDP)	UDP Broadcast	L3	Networking Core
54	Multicast stock updates to many clients	UDP Multicast	L4	Advanced Backend
55	Heartbeat on persistent TCP connection	TCP Keep-Alive / Heartbeat	L3	Backend Core
56	Allow quick server restart (SO_REUSEADDR)	Socket Option Tuning	L3	Backend Core
57	Stream API responses instead of buffering all	Streaming HTTP Response	L3	Backend Core
58	Use ETag/Last-Modified for client caching	Client-Side HTTP Cache	L2	Backend Core
59	Set connect/read timeouts for slow servers	HTTP Client Timeouts	L2	Backend Core
60	Handle HTTP redirects manually (validate Location)	Redirect Logic / Validation	L2	Backend Core
61	Async HTTP calls with Java 11 HttpClient	HttpClient sendAsync Usage	L3	Backend Core
62	Restrict TLS versions/ciphers (e.g., TLS 1.2 only)	SSLContext Config / TLS Policy	L4	Secure Backend
63	List all network interfaces + MAC addresses	NetworkInterface API	L2	Networking Core
64	Design robust binary protocol (len + CRC)	Custom Binary Protocol	L4	Advanced Backend
65	Tune TCP options (TCP_NODELAY, buffers)	Socket Performance Tuning	L3	Advanced Backend
66	WebSocket client using Java 11 HttpClient	WebSocket Client API	L3	Full Stack Backend
67	Make HTTP/2 multiplexed requests	HTTP/2 Implementation	L4	Advanced Backend

68	Call gRPC service with Java stubs & protos	gRPC Client Integration	L4	Advanced Backend
69	Zero-copy file transfer (FileChannel.transferTo)	NIO / FileChannel Performance	L4	Backend Core
70	Secure WebSocket with mutual TLS	WebSocket + TLS Client Auth	L4	Security Backend
71	Non-blocking UDP with DatagramChannel + Selector	Java NIO DatagramChannel	L4	Advanced Backend
72	Simulate many clients for load testing	Load Simulation / Concurrency	L3	Backend Core
73	Detect and close idle TCP connections (server)	Idle Connection Detection	L3	Backend Core
74	Stream file upload with Java 11 HttpClient	Streaming File Upload	L3	Backend Core
75	gRPC bidirectional streaming client	gRPC Bi-Directional Streams	L4	Advanced Backend

Scenario 1: Logistics tracking server reachability (IP:port)

Imagine a logistics company that has delivery trucks sending their location updates to a central tracking server. Before a truck's onboard system starts sending data, it needs to confirm the tracking server is alive and actually listening on the configured IP and port. If the port is closed or the server is down, sending payloads would just waste resources.

Answer:

• The reliable way is to attempt a TCP connection to the server's IP and port with a timeout.
• If the connection handshake succeeds, the port is open. If it fails or times out, the server isn't reachable.

Code:

```
import java.net.*;
import java.io.*;

public class PortProbe {
    public static boolean isPortOpen(String host, int port, int timeoutMillis) {
        SocketAddress remote = new InetSocketAddress(host, port);
        try (Socket socket = new Socket()) {
            socket.connect(remote, timeoutMillis);
            return true;
        } catch (IOException e) {
            return false;
        }
    }

    public static void main(String[] args) {
        String host = "192.168.0.50";
        int port = 5000;
        boolean open = isPortOpen(host, port, 1000);
        System.out.println(open ? "Server is up" : "Server not reachable");
    }
}
```

Reasoning to interviewer:

• A TCP connect is the most reliable indicator that something is listening on that port.
• This avoids false positives that happen with ICMP "ping," since many networks block it.

• Timeouts are critical — otherwise, one unreachable server could block the calling thread.

Future scope:

• Add retries with exponential backoff.

• Record connection success/failure metrics for monitoring.

• For TLS-based systems, extend to an SSL handshake to confirm certificates.

Scenario 2: Multiplayer game lobby with multiple players

Think of an online chess lobby. Dozens of players may join at once, chat, and wait for opponents. The server must accept multiple connections and ensure one player's slow internet doesn't freeze others.

Answer:

• Use a ServerSocket to listen on a port.
• For every incoming connection, assign it to a handler thread so it can communicate independently.
• To scale, use a thread pool (via ExecutorService) instead of creating unbounded threads.

Code:

```
import java.io.*;
import java.net.*;
import java.util.concurrent.*;

public class GameLobbyServer {
    private static final int PORT = 7000;
    private static final ExecutorService pool = Executors.newFixedThreadPool(50);

    public static void main(String[] args) {
        try (ServerSocket server = new ServerSocket(PORT)) {
            System.out.println("Game lobby started on port " + PORT);

            while (true) {
                Socket player = server.accept();
                pool.execute(new PlayerHandler(player));
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

class PlayerHandler implements Runnable {
    private final Socket playerSocket;

    public PlayerHandler(Socket socket) {
        this.playerSocket = socket;
    }

    @Override
    public void run() {
        try (BufferedReader reader = new BufferedReader(new
InputStreamReader(playerSocket.getInputStream()));
```

```
            PrintWriter writer = new PrintWriter(playerSocket.getOutputStream(), true))
{

            writer.println("Welcome to the game lobby!");
            String msg;
            while ((msg = reader.readLine()) != null) {
                System.out.println("Player says: " + msg);
                // Broadcast or handle commands
            }
        } catch (IOException e) {
            System.out.println("Player disconnected");
        }
    }
}
```

Reasoning to interviewer:

- Thread-per-client makes sense for small to medium load. For thousands of players, NIO selectors would be more scalable.
- Using a thread pool prevents resource exhaustion if too many players connect.
- By isolating each player in its own handler, slow players don't block fast ones.

Future scope:

- Add a shared broadcast system for chat.
- Implement authentication and heartbeat checks to detect dropped clients.
- For large-scale games, migrate to java.nio with non-blocking I/O.

Scenario 3: Stock price API health check

In a stock trading dashboard, prices must update every second. Before hitting the API with high-frequency requests, the app should quickly check if the server is online. Otherwise, the app should switch to cached or fallback data.

Answer:

• Send a lightweight GET request (like /health) using HttpURLConnection.
• If the response code is 200 (OK), the API is available. Otherwise, treat it as offline.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class ApiHealthCheck {
    public static void main(String[] args) {
        String apiUrl = "http://api.stockservice.com/health";

        try {
            URL url = new URL(apiUrl);
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setConnectTimeout(1000);
            conn.setReadTimeout(1000);

            int status = conn.getResponseCode();
            if (status == HttpURLConnection.HTTP_OK) {
                System.out.println("API is online and responding.");
            } else {
                System.out.println("API seems offline. Response code: " + status);
            }
        } catch (Exception e) {
            System.out.println("Error checking API: " + e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

• A health endpoint is fast and avoids unnecessary load.
• connectTimeout and readTimeout ensure the client doesn't hang waiting.
• Checking for 200 OK is enough for availability, but in production we might parse JSON for deeper checks.

Future scope:

- Add retries with backoff and circuit breaker pattern.
- Monitor response time as well as status code.
- Use libraries like Apache HttpClient or OkHttp for more advanced features.

Scenario 4: Extra encryption layer for sensitive healthcare records

In a healthcare records system, every patient report contains confidential data like diagnoses or medical history. Even though HTTPS already encrypts traffic, some organizations prefer defense-in-depth—so they want an additional encryption step. That way, if HTTPS is ever misconfigured or intercepted internally, the data is still unreadable.

Answer:

• I'd use AES (symmetric encryption) to encrypt the record before sending it over HTTPS.
• The secret AES key must be securely shared with the server (either exchanged beforehand or encrypted with RSA).
• This means even if the HTTPS layer is compromised, the payload itself remains unreadable.

Code:

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.Base64;

public class SecureUpload {
    public static void main(String[] args) {
        try {
            // Generate AES key (for demo – in production, use pre-shared key or key
exchange)
            KeyGenerator keyGen = KeyGenerator.getInstance("AES");
            SecretKey key = keyGen.generateKey();

            // Encrypt patient record
            String record = "Patient: John Doe, Diagnosis: XYZ";
            Cipher cipher = Cipher.getInstance("AES");
            cipher.init(Cipher.ENCRYPT_MODE, key);
            byte[] encrypted = cipher.doFinal(record.getBytes());
            String payload = Base64.getEncoder().encodeToString(encrypted);

            // Send encrypted payload over HTTPS
            URL url = new URL("https://hospital-server.com/upload");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("POST");
            conn.setDoOutput(true);
            conn.getOutputStream().write(payload.getBytes());

            System.out.println("Response code: " + conn.getResponseCode());
            // Server would decrypt using the shared AES key
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- AES is efficient for large payloads like medical reports.
- The HTTPS channel protects against MITM attacks, while AES payload encryption protects against insider threats or misconfigurations.
- This is similar to how some banks encrypt payloads on top of TLS for compliance.

Future scope:

- Implement RSA to exchange the AES key securely.
- Rotate encryption keys periodically for security.
- Store keys in a secure vault (like HashiCorp Vault) instead of hardcoding.

Scenario 5: Automatic TCP reconnection for trading app

In a stock trading system, the client must stay connected to the exchange server to receive live prices and send trades. But sometimes the server is slow to accept connections, or a firewall delays availability. If the client tries once and fails, users could lose trading opportunities.

Answer:

• I'll wrap the connection logic inside a retry loop.
• Each time a connection attempt fails, I'll wait a few seconds before retrying.
• Once connected, I'll break the loop and continue normal trading operations.

Code:

```
import java.net.InetSocketAddress;
import java.net.Socket;

public class StockClientReconnect {
    public static void main(String[] args) {
        String host = "stocks.example.com";
        int port = 9090;
        int retryDelay = 4000; // 4 seconds

        while (true) {
            try (Socket socket = new Socket()) {
                socket.connect(new InetSocketAddress(host, port), 3000);
                System.out.println("Connected to trading server!");
                break; // Exit loop once connected
            } catch (Exception e) {
                System.out.println("Connection failed. Retrying in " + (retryDelay / 1000)
+ "s...");
                try { Thread.sleep(retryDelay); } catch (InterruptedException ignored) {}
            }
        }
    }
}
```

Reasoning to interviewer:

• Using Socket#connect with a timeout ensures we don't hang indefinitely.
• The retry loop makes the client resilient to temporary outages.
• This is very common in trading systems where missing connection for even a few seconds can mean missing trades.

Future scope:

- Add exponential backoff with max retries to avoid hammering the server.
- Integrate logging/alerts if the server is down too long.
- Switch to a backup server if primary is unreachable.

Scenario 6: WebSocket reconnection for real-time sports scoreboard

In a live scoreboard app, users expect instant updates when a goal is scored. The server pushes events over WebSockets. If the network drops (say, user switches from Wi-Fi to mobile data), the connection breaks. If we don't auto-reconnect, users will miss live scores.

Answer:

• I'd use Java's WebSocket API with a client endpoint.
• Implement @OnClose and @OnError callbacks to detect disconnections.
• When triggered, try reconnecting with a small delay.

Code:

```
import javax.websocket.*;
import java.net.URI;

@ClientEndpoint
public class ScoreboardClient {
    private final URI serverUri = URI.create("ws://livescores.com/updates");

    public ScoreboardClient() {
        connect();
    }

    @OnOpen
    public void onOpen(Session session) {
        System.out.println("Connected to live score server.");
    }

    @OnClose
    public void onClose(Session session) {
        System.out.println("Connection closed. Retrying...");
        reconnect();
    }

    @OnError
    public void onError(Session session, Throwable t) {
        System.out.println("Error: " + t.getMessage());
        reconnect();
    }

    private void connect() {
        try {
            WebSocketContainer container =
ContainerProvider.getWebSocketContainer();
            container.connectToServer(this, serverUri);
        } catch (Exception e) {
```

```
            System.out.println("Reconnect attempt failed.");
            try { Thread.sleep(5000); } catch (InterruptedException ignored) {}
            reconnect();
        }
    }

    private void reconnect() {
        connect();
    }

    public static void main(String[] args) {
        new ScoreboardClient();
    }
}
```

Reasoning to interviewer:

- WebSockets are long-lived, so network interruptions are expected. Automatic reconnection improves UX.
- Using callbacks (@OnClose, @OnError) ensures the app reacts immediately to failures.
- Adding delay avoids rapid reconnect loops that can flood the server.

Future scope:

- Use exponential backoff or capped retry intervals.
- Re-subscribe to channels (e.g., specific teams) after reconnecting.
- Log metrics on disconnections to detect recurring network issues.

Scenario 7: Handling HTTP redirects in a travel booking app

In a travel booking platform, you might first hit a generic domain like http://booking.example.com. That server may redirect you to a regional cluster, say https://asia.booking.com, for better latency or compliance. If the client doesn't follow redirects, users may get errors or incomplete responses.

Answer:

• By default, HttpURLConnection can follow redirects using setInstanceFollowRedirects(true).
• If you want custom behavior (like limiting to same domain), you can check for 3xx codes and manually read the Location header to reconnect.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class RedirectExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://booking.example.com/redirect");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setInstanceFollowRedirects(true); // auto redirect

            int code = conn.getResponseCode();
            if (code / 100 == 3) {
                String newUrl = conn.getHeaderField("Location");
                System.out.println("Redirected to: " + newUrl);
            } else {
                System.out.println("Response Code: " + code);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Some APIs require the client to explicitly follow redirects, otherwise you'll only see a 301 or 302.
• Automatic redirect works for most cases, but sometimes you need to validate the new domain before connecting (security best practice).
• Handling this manually allows us to prevent open redirect vulnerabilities.

Future scope:

- Limit the number of redirects (to avoid infinite loops).
- Apply domain whitelisting before following the redirect.
- Use higher-level libraries like Apache HttpClient or OkHttp that provide more redirect options.

Scenario 8: Different proxies for different APIs in analytics

Suppose you have an analytics service that calls both a payments API and a shipments API. Compliance rules say payments must go through one proxy, while shipments must use another. A single global JVM proxy setting won't work here.

Answer:

• Instead of relying on system properties, I'd create Proxy objects for each API call.
• HttpURLConnection allows opening a connection with a specific proxy.

Code:

```
import java.net.*;

public class MultipleProxiesExample {
    public static void main(String[] args) {
        try {
            // Payments API through proxy1
            URL api1 = new URL("http://payments.example.com/data");
            Proxy proxy1 = new Proxy(Proxy.Type.HTTP,
                    new InetSocketAddress("proxy1.example.com", 8080));
            HttpURLConnection conn1 = (HttpURLConnection)
api1.openConnection(proxy1);
            System.out.println("Payments API Response: " +
conn1.getResponseCode());

            // Shipments API through proxy2
            URL api2 = new URL("http://shipments.example.com/data");
            Proxy proxy2 = new Proxy(Proxy.Type.HTTP,
                    new InetSocketAddress("proxy2.example.com", 9090));
            HttpURLConnection conn2 = (HttpURLConnection)
api2.openConnection(proxy2);
            System.out.println("Shipments API Response: " +
conn2.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• This approach is flexible — each connection can have its own proxy settings.
• It's useful when integrating with multiple partners who require routing

through their own gateways.

• It avoids the limitation of a single global JVM proxy.

Future scope:

• Add authentication for proxies (via Authenticator).

• Switch to a connection pooling library like OkHttp for efficiency.

• Log which proxy was used for audit purposes.

Scenario 9: Debugging SSL handshake in a banking application

In a banking system, the client must establish secure SSL/TLS connections. If the handshake fails, it may be due to expired certificates, unsupported cipher suites, or protocol mismatches. To debug, developers often need to see the handshake details.

Answer:

• In Java, enabling SSL debug logs is as simple as setting javax.net.debug=ssl.
• This prints certificate chains, chosen protocols, and cipher suites to stdout.

Code:

```
import javax.net.ssl.HttpsURLConnection;
import java.net.URL;

public class SSLHandshakeLogger {
    static {
        System.setProperty("javax.net.debug", "ssl");
    }

    public static void main(String[] args) {
        try {
            URL url = new URL("https://secure.bankserver.com");
            HttpsURLConnection conn = (HttpsURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            System.out.println("Response Code: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Debug logs reveal certificate chains, whether the server presented the correct intermediate certs, and which cipher was negotiated.
• This is critical in banking systems where TLS versions and ciphers must meet strict compliance.
• It's also useful for diagnosing handshake failures during mutual TLS setups.

Future scope:

• Narrow the debug level, e.g., javax.net.debug=ssl:handshake for just handshake details.

• Capture logs into a file for analysis.

• Use SSLParameters to enforce stronger ciphers once debugging is done.

Scenario 10: Live cricket score server that streams updates

When a client connects to the score server, they should get a greeting and then periodic score pushes (every few seconds). Think of it like a radio broadcast—connection stays open and we keep writing incremental updates.

Answer:

• I'll keep a ServerSocket listening and hand each client to a handler so one slow client doesn't block others.
• Each handler writes a greeting, then enters a loop to push updates, flushing the writer each time so the client doesn't buffer indefinitely.
• I'll add a small protocol (newline-delimited text) and graceful shutdown handling.

Code:

```
import java.io.*;
import java.net.*;
import java.time.LocalTime;
import java.util.Random;
import java.util.concurrent.*;

public class StreamingServer {
    private static final int PORT = 7070;
    private static final ExecutorService pool = Executors.newCachedThreadPool();

    public static void main(String[] args) {
        System.out.println("Streaming server starting on port " + PORT);
        try (ServerSocket server = new ServerSocket(PORT)) {
            while (true) {
                Socket client = server.accept();
                client.setTcpNoDelay(true);
                pool.execute(new ClientStreamer(client));
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            pool.shutdown();
        }
    }

    static class ClientStreamer implements Runnable {
        private final Socket socket;
        private final Random rand = new Random();

        ClientStreamer(Socket socket) { this.socket = socket; }

        @Override
        public void run() {
            try (var out = new PrintWriter(new BufferedWriter(
```

```
                new OutputStreamWriter(socket.getOutputStream())), true)) {
            out.println("Welcome to live cricket scores!");
            int runs = 0, wickets = 0;

            while (!socket.isClosed()) {
                runs += rand.nextInt(7); // 0..6
                if (rand.nextDouble() < 0.05 && wickets < 10) wickets++;
                String line = "[%s] Score update: %d/%d".formatted(
                        LocalTime.now(), runs, wickets);
                out.println(line); // auto-flush = true
                Thread.sleep(3000);
            }
        } catch (IOException | InterruptedException e) {
            // client disconnected or server shutting down
        } finally {
            try { socket.close(); } catch (IOException ignored) {}
        }
    }
  }
}
```

Reasoning to interviewer:

• Thread-per-connection with a cached pool keeps the code simple and responsive for dozens/hundreds of clients. For thousands+, I'd switch to NIO (Selector).

• PrintWriter with autoflush guarantees each update is pushed immediately, which matters for "live" feels.

• Keeping a small protocol (one update per line) makes it easy for clients to parse and display.

Future scope:

• Replace random updates with a publisher/subscriber channel, so multiple handlers consume the same live feed.

• Add heartbeats and client timeouts to prune dead connections.

• Consider HTTP chunked responses or Server-Sent Events (SSE) if you want browser-friendly streaming.

Scenario 11: Serialize/deserialize JSON to exchange user data

Your app sends Java objects to a REST API as JSON and reads JSON responses back. In practice, you'll want stable field names, null handling, and forward-compatibility when fields are added later.

Answer:

• I'll use Jackson's ObjectMapper—it's fast, widely adopted, and handles most needs out of the box.
• For robustness, I'll ignore unknown fields (helps when the server adds fields) and use records/POJOs with getters.
• Example shows both directions: object → JSON and JSON → object.

Code:

```
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.ObjectMapper;

// Domain model
@JsonIgnoreProperties(ignoreUnknown = true) // tolerate extra fields from server
class User {
    @JsonProperty("name") private String name;
    @JsonProperty("age")  private int age;

    public User() {} // needed for deserialization
    public User(String name, int age) { this.name = name; this.age = age; }

    public String getName() { return name; }
    public int getAge() { return age; }
}

public class JsonExample {
    public static void main(String[] args) {
        try {
            ObjectMapper mapper = new ObjectMapper()

.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);

            // Serialize Java object → JSON text
            User user = new User("Alice", 25);
            String json = mapper.writeValueAsString(user);
            System.out.println("JSON: " + json);

            // Deserialize JSON text → Java object
            String incoming = """{"name":"Bob","age":31,"extra":"ignored"}""";
            User user2 = mapper.readValue(incoming, User.class);
```

```
            System.out.println("Deserialized: " + user2.getName() + ", " +
user2.getAge());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- @JsonIgnoreProperties(ignoreUnknown=true) plus the mapper config lets the client survive server-side field additions—important for backward compatibility.
- Keeping a no-args constructor and getters makes POJOs deserialization-friendly.
- If I need immutability, I'd use Java records and Jackson's record support.

Future scope:

- Enable field naming strategies (e.g., snake_case) to match API conventions.
- Add validation (e.g., Bean Validation) after deserialization to catch bad payloads early.
- Use streaming (JsonParser/JsonGenerator) for very large payloads to reduce memory pressure.

Scenario 12: Monitor multiple URLs and log their HTTP status

A monitoring tool pings endpoints and records whether they're healthy. Here we want to categorize outcomes (OK, Not Found, Server Error, etc.) and avoid hanging when a site is slow.

Answer:

• I'll open a HttpURLConnection per URL, set short connect/read timeouts, call getResponseCode(), and map it to a human-friendly status.
• For observability, I'll include timing and exceptions in logs. A HEAD request is lighter if the server supports it.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.Map;

public class HttpStatusLogger {
    public static void main(String[] args) {
        String[] urls = {
            "http://site.com/success",
            "http://site.com/missing",
            "http://site.com/error"
        };

        for (String u : urls) {
            long start = System.currentTimeMillis();
            try {
                HttpURLConnection conn = (HttpURLConnection) new
URL(u).openConnection();
                conn.setRequestMethod("GET"); // or "HEAD" if supported
                conn.setConnectTimeout(1500);
                conn.setReadTimeout(2000);

                int code = conn.getResponseCode();
                String category = categorize(code);
                long ms = System.currentTimeMillis() - start;

                System.out.printf("%s - %s (%d) in %d ms%n", u, category, code, ms);
            } catch (Exception e) {
                long ms = System.currentTimeMillis() - start;
                System.out.printf("%s - FAILURE (%s) in %d ms%n", u,
e.getClass().getSimpleName(), ms);
            }
        }
    }

    private static String categorize(int code) {
```

```
        if (code >= 200 && code < 300) return "Success";
        if (code >= 300 && code < 400) return "Redirect";
        if (code == HttpURLConnection.HTTP_NOT_FOUND) return "Not Found";
        if (code >= 400 && code < 500) return "Client Error";
        if (code >= 500) return "Server Error";
        return "Unknown";
    }
}
```

Reasoning to interviewer:

- Timeouts prevent a single bad endpoint from stalling the whole run—critical for reliable monitors.
- Categorizing by ranges (2xx/3xx/4xx/5xx) gives quick health signals; calling out 404 separately helps catch broken routes.
- Logging elapsed time helps detect performance regressions, not just availability.

Future scope:

- Add retries with backoff for transient 5xx/timeout errors.
- Emit metrics to a time-series DB (Prometheus) and alert on SLO breaches.
- Parallelize with a thread pool to check many URLs quickly while capping concurrency.

Scenario 13: Show upload progress for large file backups

In a cloud backup tool, a 5–10 GB archive may take minutes to upload. Users expect a live percentage (and ideally ETA). The client should stream the file in chunks, not load it all into memory, and update progress as bytes are written.

Answer:

• I'll open an HttpURLConnection in POST mode and stream the file in chunks from disk to the request body; no full buffering.
• I'll either set setFixedLengthStreamingMode(fileSize) (if known) or setChunkedStreamingMode so the connection doesn't buffer the whole file.
• While copying, I'll update uploaded/total and print percent; the same hook can publish to a UI progress bar.

Code:

```
import java.io.*;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.file.Files;
import java.time.Duration;
import java.time.Instant;

public class FileUploadProgress {
    public static void main(String[] args) {
        File file = new File("backup.zip");
        long total = file.length();

        try {
            URL url = new URL("http://cloud.example.com/upload");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("POST");
            conn.setDoOutput(true);
            conn.setConnectTimeout(5000);
            conn.setReadTimeout(30000);

            // Advertise known length so server can validate integrity
            conn.setFixedLengthStreamingMode(total);
            conn.setRequestProperty("Content-Type", "application/octet-stream");

            byte[] buffer = new byte[8192];
            long uploaded = 0;
            Instant start = Instant.now();

            try (OutputStream out = new
BufferedOutputStream(conn.getOutputStream(), 8192);
                 InputStream in = new
BufferedInputStream(Files.newInputStream(file.toPath()), 8192)) {
```

```
                int read;
                while ((read = in.read(buffer)) != -1) {
                    out.write(buffer, 0, read);
                    uploaded += read;

                    double percent = uploaded * 100.0 / total;
                    long elapsedMs = Duration.between(start, Instant.now()).toMillis();
                    double speed = (uploaded / 1024.0 / 1024.0) / (elapsedMs / 1000.0 +
1e-6); // MB/s
                    System.out.printf("Uploaded: %.2f%% (%.2f MB/s)%n", percent,
speed);
                }
                out.flush();
            }

            int code = conn.getResponseCode();
            System.out.println("Upload finished. Response: " + code);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- Fixed-length streaming avoids buffering entire payloads and lets servers enforce Content-Length. For unknown sizes (e.g., on-the-fly compression), I'd use setChunkedStreamingMode(8192).
- Buffered streams reduce syscall overhead; an 8 KB–64 KB buffer is a good default.
- The progress math is simple: bytes sent / file size. I also log throughput to reassure users the upload is alive.

Future scope:

- Multipart upload (S3-style) with resumable parts and checksums.
- UI progress events instead of console logs; show ETA based on moving average speed.
- Retry strategy on transient network errors, resuming from last acknowledged byte if the server supports it.

Scenario 14: Compress JSON with GZIP before sending

Chat messages are small individually but huge in aggregate. Compressing the JSON request body can cut bandwidth and cost, especially on mobile networks.

Answer:

• I'll set Content-Encoding: gzip and wrap the request body stream with GZIPOutputStream, then write the JSON bytes.
• I'll keep Content-Type: application/json so the server routes correctly.
• Close or finish() the GZIP stream before reading the response to ensure the footer is written.

Code:

```
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.util.zip.GZIPOutputStream;

public class GzipJsonExample {
    public static void main(String[] args) {
        String json = "{\"message\":\"Hello World\",\"sender\":\"Rajat\"}";
        try {
            URL url = new URL("http://chat.example.com/send");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("POST");
            conn.setDoOutput(true);
            conn.setConnectTimeout(5000);
            conn.setReadTimeout(10000);

            conn.setRequestProperty("Content-Type", "application/json");
            conn.setRequestProperty("Content-Encoding", "gzip");

            try (OutputStream rawOut = conn.getOutputStream();
                 GZIPOutputStream gzipOut = new GZIPOutputStream(rawOut)) {
                gzipOut.write(json.getBytes(StandardCharsets.UTF_8));
                gzipOut.finish(); // make sure trailer is written
            }

            System.out.println("Response: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Declaring Content-Encoding: gzip tells the server to decompress the body before parsing JSON.

• finish() (or closing the stream) is required to flush the GZIP trailer; without it, some servers treat the body as truncated.

• If responses are also large, I can send Accept-Encoding: gzip and read via a GZIPInputStream when the server returns compressed content.

Future scope:

• Switch to a pooled HTTP client (OkHttp/Apache HttpClient) for connection reuse.

• Apply compression threshold (e.g., only gzip if payload > 1 KB) to avoid overhead on tiny messages.

• Add integrity via a checksum header (e.g., Content-MD5) if required.

Scenario 15: Parallel download of a large video with HTTP ranges

Training videos can be several GBs. To accelerate downloads on fast networks, the client can request multiple byte ranges in parallel and write them into the correct offsets of a single output file.

Answer:

• I'll first probe the server (HEAD/initial GET) for Content-Length and Accept-Ranges: bytes.
• I'll split the file into equal ranges, spawn worker tasks, and for each task set the Range header bytes=start-end.
• Each worker writes to a single RandomAccessFile at its offset (or separate temp files that we join later).

Code:

```
import java.io.*;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

public class ParallelFileDownloader {
    public static void main(String[] args) throws Exception {
        String fileURL = "http://videos.example.com/large.mp4";
        String outFile = "video.mp4";
        int threads = 4;

        DownloadPlan plan = probe(fileURL);
        if (!plan.acceptRanges) {
            System.out.println("Server does not support ranges; falling back to single-
threaded download.");
            singleDownload(fileURL, outFile);
            return;
        }

        ExecutorService pool = Executors.newFixedThreadPool(threads);
        List<Future<?>> futures = new ArrayList<>();

        // Preallocate target file
        try (RandomAccessFile raf = new RandomAccessFile(outFile, "rw")) {
            raf.setLength(plan.length);
        }

        long partSize = plan.length / threads;
        for (int i = 0; i < threads; i++) {
            long start = i * partSize;
            long end = (i == threads - 1) ? (plan.length - 1) : (start + partSize - 1);
```

```
            futures.add(pool.submit(new PartTask(fileURL, outFile, start, end, i)));
        }

        for (Future<?> f : futures) f.get(); // wait for all
        pool.shutdown();
        System.out.println("Download complete: " + outFile);
    }

    static class DownloadPlan {
        long length;
        boolean acceptRanges;
    }

    private static DownloadPlan probe(String urlStr) throws IOException {
        HttpURLConnection c = (HttpURLConnection) new
URL(urlStr).openConnection();
        c.setRequestMethod("HEAD");
        c.setConnectTimeout(5000);
        c.setReadTimeout(5000);
        int code = c.getResponseCode();
        DownloadPlan p = new DownloadPlan();
        p.length = c.getContentLengthLong();
        String ar = c.getHeaderField("Accept-Ranges");
        p.acceptRanges = (ar != null && ar.toLowerCase().contains("bytes"));
        System.out.printf("Probe -> code=%d, length=%d, acceptRanges=%s%n",
code, p.length, p.acceptRanges);
        return p;
    }

    private static void singleDownload(String urlStr, String outFile) throws
IOException {
        HttpURLConnection c = (HttpURLConnection) new
URL(urlStr).openConnection();
        c.setConnectTimeout(5000);
        c.setReadTimeout(30000);
        try (InputStream in = new BufferedInputStream(c.getInputStream());
             OutputStream out = new BufferedOutputStream(new
FileOutputStream(outFile))) {
            byte[] buf = new byte[8192];
            int n;
            while ((n = in.read(buf)) != -1) out.write(buf, 0, n);
        }
    }

    static class PartTask implements Runnable {
        private final String urlStr, outFile;
        private final long start, end;
        private final int partId;

        PartTask(String urlStr, String outFile, long start, long end, int partId) {
```

```
            this.urlStr = urlStr; this.outFile = outFile;
            this.start = start; this.end = end; this.partId = partId;
        }

        @Override public void run() {
            try {
                HttpURLConnection c = (HttpURLConnection) new
URL(urlStr).openConnection();
                c.setRequestProperty("Range", "bytes=" + start + "-" + end);
                c.setConnectTimeout(5000);
                c.setReadTimeout(30000);

                int code = c.getResponseCode();
                if (code != HttpURLConnection.HTTP_PARTIAL && code !=
HttpURLConnection.HTTP_OK) {
                    throw new IOException("Unexpected HTTP " + code + " for part " +
partId);
                }

                try (InputStream in = new BufferedInputStream(c.getInputStream());
                     RandomAccessFile raf = new RandomAccessFile(outFile, "rw")) {
                    raf.seek(start);
                    byte[] buf = new byte[8192];
                    int n;
                    while ((n = in.read(buf)) != -1) {
                        raf.write(buf, 0, n);
                    }
                }
                System.out.printf("Part %d [%d-%d] downloaded.%n", partId, start, end);
            } catch (Exception e) {
                System.err.printf("Part %d failed: %s%n", partId, e.getMessage());
                throw new RuntimeException(e);
            }
        }
    }
}
```

Reasoning to interviewer:

- Probing for Accept-Ranges: bytes prevents wasted parallel attempts on servers that don't support ranges.
- RandomAccessFile#setLength preallocates space, avoiding sparse-file surprises and allowing workers to write safely to offsets.
- Validating the HTTP status (206 Partial Content) ensures we're actually receiving the requested slice.

Future scope:

- Add retries per part, with checksum validation (e.g., MD5/SHA-256 for each chunk).
- Adaptive chunk sizing: start with small parts, increase if network is strong.
- Merge to a single pooled HTTP client for connection reuse, TLS warmup, and better timeout controls.

Scenario 16: Real-time stock prices over Server-Sent Events (SSE)

In a trading dashboard, the backend publishes a never-ending stream like text/event-stream with lines such as data:{...price...}. The client keeps one HTTP connection open and processes each event as it arrives—no polling, just a steady feed.

Answer:

• I'll open an HTTP GET, set reasonable timeouts, and keep reading the response line-by-line.
• I'll parse SSE fields (event:, id:, retry:, data:). When I see data:, I'll accumulate until a blank line (end of event), then process the payload.
• If the stream breaks, I'll reconnect (optionally using the last id via Last-Event-ID).

Code:

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.time.Duration;

public class SSEClient {
    private static final String URL_STR = "http://stocks.example.com/stream";
    private static final int CONNECT_TIMEOUT_MS = 4000;
    private static final int READ_TIMEOUT_MS = 65_000; // > server heartbeat

    public static void main(String[] args) {
        new SSEClient().run();
    }

    private void run() {
        String lastEventId = null;

        while (true) {
            HttpURLConnection conn = null;
            try {
                URL url = new URL(URL_STR);
                conn = (HttpURLConnection) url.openConnection();
                conn.setRequestMethod("GET");
                conn.setRequestProperty("Accept", "text/event-stream");
                conn.setRequestProperty("Cache-Control", "no-cache");
                if (lastEventId != null) conn.setRequestProperty("Last-Event-ID",
lastEventId);
                conn.setConnectTimeout(CONNECT_TIMEOUT_MS);
                conn.setReadTimeout(READ_TIMEOUT_MS);

                try (BufferedReader br = new BufferedReader(
                        new InputStreamReader(conn.getInputStream(),
StandardCharsets.UTF_8))) {
```

```
                StringBuilder dataBlock = new StringBuilder();
                String line;
                while ((line = br.readLine()) != null) {
                    if (line.isEmpty()) { // end of one SSE event
                        if (dataBlock.length() > 0) {
                            String data = dataBlock.toString();
                            System.out.println("Stock update: " + data);
                            dataBlock.setLength(0);
                        }
                        continue;
                    }
                    if (line.startsWith("id:")) {
                        lastEventId = line.substring(3).trim();
                    } else if (line.startsWith("data:")) {
                        if (dataBlock.length() > 0) dataBlock.append('\n');
                        dataBlock.append(line.substring(5).trim());
                    }
                    // handle event:/retry: if needed
                }
            }
            // server closed cleanly; loop will reconnect
        } catch (IOException e) {
            System.out.println("SSE connection lost: " + e.getMessage());
        } finally {
            if (conn != null) conn.disconnect();
        }

        sleepQuietly(Duration.ofSeconds(3)); // small backoff before reconnect
    }
  }

  private static void sleepQuietly(Duration d) {
    try { Thread.sleep(d.toMillis()); } catch (InterruptedException ignored) {}
  }
}
```

Reasoning to interviewer:

- SSE is perfect for one-way market tick streams: simpler than WebSockets and supported by proxies/CDNs.
- Parsing per the SSE spec (aggregate data: lines until blank line) ensures multi-line JSON payloads are handled.
- Keeping Last-Event-ID lets me resume after a network blip without missing ticks (if the server supports it).

Future scope:

- Add exponential backoff, jitter, and metrics (events/sec, reconnects).
- If the UI is Swing/JavaFX, post updates on the UI thread; debounce to avoid flooding.
- Fall back to polling if text/event-stream isn’t available in certain environments.

Scenario 17: Two-way chat over WebSockets (server side)

A chat room needs full-duplex: clients send messages; the server broadcasts to everyone. WebSockets give a single long-lived TCP connection where both sides can push.

Answer:

• I'll expose a @ServerEndpoint("/chat"). On open, I'll add the session to a concurrent set. On message, I'll broadcast to all connected sessions using the async remote endpoint. On close/error, I'll remove the session.
• This runs inside a WebSocket-capable container (Tomcat/Jetty/Undertow) or via an embedded server.

Code:

```
import javax.websocket.*;
import javax.websocket.server.ServerEndpoint;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

@ServerEndpoint("/chat")
public class ChatServer {
    private static final Set<Session> SESSIONS =
ConcurrentHashMap.newKeySet();

    @OnOpen
    public void onOpen(Session session) {
        SESSIONS.add(session);
        session.getAsyncRemote().sendText("Welcome! Your id: " + session.getId());
        System.out.println("Client connected: " + session.getId());
    }

    @OnMessage
    public void onMessage(String message, Session session) {
        String payload = "[" + session.getId() + "]: " + message;
        for (Session s : SESSIONS) {
            if (s.isOpen()) s.getAsyncRemote().sendText(payload);
        }
    }

    @OnClose
    public void onClose(Session session, CloseReason reason) {
        SESSIONS.remove(session);
        System.out.println("Client disconnected: " + session.getId() + " (" + reason +
")");
    }

    @OnError
    public void onError(Session session, Throwable t) {
```

```
        System.err.println("Error for " + (session != null ? session.getId() : "n/a") + ": "
+ t);
    }
}
```

Reasoning to interviewer:

• Using ConcurrentHashMap.newKeySet() avoids locking and scales for many clients.

• Async sends (getAsyncRemote()) prevent a slow client from blocking the broadcast loop.

• The endpoint is stateless; the only shared state is the session set, which makes horizontal scaling easier (with a pub/sub backplane if we shard nodes).

Future scope:

• Add rooms/topics; persist recent messages; authenticate users and rate-limit.

• Scale out with a message broker (Redis pub/sub or Kafka) so all nodes broadcast consistently.

• Add heartbeats (ping/pong) and close idle sessions to reclaim resources.

Scenario 18: Download a protected report with HTTP Basic Auth

Some internal services protect static files with Basic Auth. The client must send Authorization: Basic base64(user:pass) on the request.

Answer:

• I'll build user:password, encode with Base64 (UTF-8), and set it on the Authorization header.
• I'll prefer HTTPS so credentials aren't exposed on the wire. For repeated use, Authenticator is cleaner.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.util.Base64;

public class BasicAuthExample {
    public static void main(String[] args) {
        String urlStr = "https://reports.example.com/securefile";
        String user = "user123";
        String pass = "pass123";

        try {
            String creds = user + ":" + pass;
            String encoded =
Base64.getEncoder().encodeToString(creds.getBytes(StandardCharsets.UTF_8));

            URL url = new URL(urlStr);
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setConnectTimeout(4000);
            conn.setReadTimeout(15000);
            conn.setRequestProperty("Authorization", "Basic " + encoded);

            int code = conn.getResponseCode();
            System.out.println("Response Code: " + code);
            // read body if needed
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Basic Auth is just a header; it’s not encryption—hence HTTPS is non-negotiable.

• Using StandardCharsets.UTF_8 ensures consistent encoding of credentials.

• For multiple requests, Authenticator.setDefault(...) reduces duplication; for modern APIs, I’d move to OAuth or signed URLs.

Future scope:

• Use Authenticator or a pooled HTTP client (OkHttp/Apache) for connection reuse and automatic auth.

• Store secrets in a secure vault; rotate credentials regularly.

• Add retry/backoff on transient failures and verify TLS settings (protocols/ciphers) for compliance.

Scenario 19: High-concurrency stock feed server with non-blocking I/O

A stock feed has thousands of concurrent subscribers. A classic thread-per-client server wastes memory and context switches. With Java NIO, one (or a few) selector threads can multiplex accept/read/write for all sockets.

Answer:

• I'll use ServerSocketChannel in non-blocking mode and a Selector.
• Each new SocketChannel is also non-blocking; I register it with OP_READ and attach a small per-client buffer/state.
• The selector loop checks ready keys; when readable, I read into a buffer and queue a response (or broadcast) by setting OP_WRITE interest.
• This design scales to thousands of sockets with a handful of threads.

Code:

```
import java.net.InetSocketAddress;
import java.nio.ByteBuffer;
import java.nio.channels.*;
import java.util.Iterator;
import java.util.Set;
import java.util.concurrent.ConcurrentLinkedQueue;

public class NonBlockingServer {
    static class ClientState {
        final ByteBuffer readBuf = ByteBuffer.allocate(8 * 1024);
        final ConcurrentLinkedQueue<ByteBuffer> outbound = new
ConcurrentLinkedQueue<>();
    }

    public static void main(String[] args) {
        try (Selector selector = Selector.open();
             ServerSocketChannel server = ServerSocketChannel.open()) {

            server.bind(new InetSocketAddress(8080));
            server.configureBlocking(false);
            server.register(selector, SelectionKey.OP_ACCEPT);

            System.out.println("NIO server on :8080");

            while (true) {
                selector.select(); // block until at least one key is ready
                Set<SelectionKey> selected = selector.selectedKeys();
                for (Iterator<SelectionKey> it = selected.iterator(); it.hasNext(); ) {
                    SelectionKey key = it.next();
                    it.remove();

                    try {
```

```
                if (key.isValid() && key.isAcceptable()) {
                    ServerSocketChannel ssc = (ServerSocketChannel)
key.channel();
                    SocketChannel client = ssc.accept();
                    if (client == null) continue;
                    client.configureBlocking(false);
                    SelectionKey ck = client.register(selector,
SelectionKey.OP_READ);
                    ck.attach(new ClientState());
                    System.out.println("Client connected: " +
client.getRemoteAddress());
                }

                if (key.isValid() && key.isReadable()) {
                    SocketChannel ch = (SocketChannel) key.channel();
                    ClientState st = (ClientState) key.attachment();
                    int n = ch.read(st.readBuf);
                    if (n == -1) {
                        closeSilently(key, ch);
                        continue;
                    }
                    if (n > 0) {
                        st.readBuf.flip();
                        // Echo or parse request; here we queue a fake price tick
                        byte[] in = new byte[st.readBuf.remaining()];
                        st.readBuf.get(in);
                        st.readBuf.clear();

                        String tick = "PRICE:" + System.currentTimeMillis() + "\n";
                        st.outbound.add(ByteBuffer.wrap(tick.getBytes()));
                        key.interestOps(key.interestOps() | SelectionKey.OP_WRITE);
                    }
                }

                if (key.isValid() && key.isWritable()) {
                    SocketChannel ch = (SocketChannel) key.channel();
                    ClientState st = (ClientState) key.attachment();

                    ByteBuffer buf;
                    while ((buf = st.outbound.peek()) != null) {
                        ch.write(buf);
                        if (buf.hasRemaining()) break;   // socket backpressure; try
later
                        st.outbound.poll();              // fully sent
                    }
                    if (st.outbound.isEmpty()) {
                        key.interestOps(key.interestOps() &
~SelectionKey.OP_WRITE);
                    }
                }
```

```
                } catch (Exception ex) {
                    closeSilently(key, (SelectableChannel) key.channel());
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private static void closeSilently(SelectionKey key, SelectableChannel ch) {
    try { key.cancel(); ch.close(); } catch (Exception ignored) {}
}
}
```

Reasoning to interviewer:

- Selector-driven I/O lets one thread manage thousands of sockets; CPU scales with real activity, not connection count.
- Per-client state is kept tiny (read buffer + outbound queue), so memory stays predictable.
- Toggling OP_WRITE only when you actually have bytes to send avoids busy loops.
- Backpressure is natural: if a channel can't accept more bytes, we keep the remaining buffer and retry when writable.

Future scope:

- Split acceptor and I/O workers (multi-reactor) on multi-core boxes.
- Add a ring buffer or disruptor queue to ingest market ticks and fan them out to interested channels.
- Use direct ByteBuffers for zero-copy writes and gathering write (Scattering/Gathering) for batching.
- TLS offload: either SSLEngine over NIO or a terminating proxy (e.g., nginx) in front.

Scenario 20: File transfer between two machines over sockets

You need a simple LAN file-share: one Java process serves, the other sends. The protocol should be robust (know the filename and length) and stream in chunks to avoid loading the whole file in memory.

Answer:

• I'll define a tiny framing protocol: first send the filename length + filename, then a 64-bit file size, followed by raw bytes.
• Server reads headers, pre-creates the target file, then streams bytes to disk.
• Client reads file in chunks and writes to the socket; both sides use buffered streams and try-with-resources.

Code (Server):

```
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class FileServer {
    public static void main(String[] args) {
        try (ServerSocket server = new ServerSocket(9090)) {
            System.out.println("Waiting for file on :9090 ...");
            while (true) {
                try (Socket client = server.accept();
                     DataInputStream in = new DataInputStream(new
BufferedInputStream(client.getInputStream()))) {

                    int nameLen = in.readInt();
                    byte[] nameBytes = in.readNBytes(nameLen);
                    String fileName = new String(nameBytes, StandardCharsets.UTF_8);

                    long fileSize = in.readLong();
                    System.out.printf("Receiving %s (%d bytes)%n", fileName, fileSize);

                    try (OutputStream fileOut = new BufferedOutputStream(new
FileOutputStream(fileName))) {
                        byte[] buf = new byte[8192];
                        long remaining = fileSize;
                        while (remaining > 0) {
                            int read = in.read(buf, 0, (int) Math.min(buf.length, remaining));
                            if (read == -1) throw new EOFException("Unexpected end of
stream");
                            fileOut.write(buf, 0, read);
                            remaining -= read;
                        }
                    }
                    System.out.println("File received successfully: " + fileName);
```

```
            } catch (Exception ex) {
                System.err.println("Transfer failed: " + ex.getMessage());
            }
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
  }
}
```

Code (Client):

```
import java.io.*;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class FileClient {
    public static void main(String[] args) {
        String host = "192.168.1.10"; // receiver IP
        int port = 9090;
        File toSend = new File("send.txt");

        try (Socket socket = new Socket(host, port);
             DataOutputStream out = new DataOutputStream(new
BufferedOutputStream(socket.getOutputStream()));
             InputStream fileIn = new BufferedInputStream(new
FileInputStream(toSend))) {

            byte[] nameBytes =
toSend.getName().getBytes(StandardCharsets.UTF_8);
            out.writeInt(nameBytes.length);
            out.write(nameBytes);
            out.writeLong(toSend.length());

            byte[] buf = new byte[8192];
            int n;
            while ((n = fileIn.read(buf)) != -1) {
                out.write(buf, 0, n);
            }
            out.flush();
            System.out.println("File sent successfully.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Using a tiny header (filename length + bytes + file size) makes the protocol self-describing and prevents the server from guessing when the file ends.
• DataInputStream/DataOutputStream simplify fixed-width primitives (int/long), reducing parsing bugs.
• Buffered I/O and 8–32 KB chunks provide good throughput without excessive heap usage.
• Strictly reading remaining bytes guards against partial writes/reads and EOF mid-transfer.

Future scope:

• Add checksum (e.g., SHA-256) after the payload for integrity verification.
• Support multiple files per connection (loop until a sentinel value).
• TLS (SSLSocket) for confidentiality on untrusted networks.
• Parallel/multipart sending for very large files with resume capability.

Scenario 21: Streaming very large JSON responses (news feed)

When a news API returns megabytes of JSON, reading it all into memory is risky. I prefer a streaming approach so memory stays flat even for huge payloads.

Answer:

• I'll open the HTTP connection with short timeouts and stream the body line-by-line (or byte-by-byte) instead of loading it all.
• For structured processing, I'll use Jackson's streaming parser (JsonFactory + JsonParser) to iterate items (e.g., "articles" array) without materializing the entire JSON.
• If I only need the first N items for the UI, I can stop parsing early.

Code:

// Option A: simple chunked read (good for logging/snippets)

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

public class LargeJsonHandler {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://api.newsportal.com/latest");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setConnectTimeout(3000);
            conn.setReadTimeout(15000);
            conn.setRequestProperty("Accept", "application/json");

            try (BufferedReader reader = new BufferedReader(
                    new InputStreamReader(conn.getInputStream(),
StandardCharsets.UTF_8), 16_384)) {

                StringBuilder head = new StringBuilder(2_000);
                String line;
                while ((line = reader.readLine()) != null && head.length() < 2_000) {
                    head.append(line).append('\n'); // stream; don't build full body in
production
                }
                System.out.println("First ~2 KB:\n" + head);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

// Option B: production-friendly streaming parse with Jackson

```
import com.fasterxml.jackson.core.*;
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

public class LargeJsonStreamingParse {
    public static void main(String[] args) {
        JsonFactory jf = new JsonFactory();
        try {
            HttpURLConnection conn = (HttpURLConnection) new
URL("http://api.newsportal.com/latest").openConnection();
            conn.setRequestProperty("Accept", "application/json");
            conn.setConnectTimeout(3000);
            conn.setReadTimeout(15000);

            try (InputStream in = new BufferedInputStream(conn.getInputStream(),
32_768);
                 JsonParser p = jf.createParser(in)) {

                // Example: root { "articles": [ { ... }, { ... } ] }
                while (p.nextToken() != null) {
                    if (p.currentToken() == JsonToken.FIELD_NAME &&
"articles".equals(p.getCurrentName())) {
                        p.nextToken(); // should be START_ARRAY
                        int shown = 0;
                        while (p.nextToken() == JsonToken.START_OBJECT && shown <
10) {
                            String title = null, author = null;
                            while (p.nextToken() != JsonToken.END_OBJECT) {
                                String name = p.getCurrentName();
                                p.nextToken();
                                if ("title".equals(name))  title = p.getValueAsString();
                                if ("author".equals(name)) author = p.getValueAsString();
                                p.skipChildren(); // skip nested structures we don't need
                            }
                            System.out.println("Article: " + title + " by " + author);
                            shown++;
                        }
                        break; // stop early if we only need first page
                    }
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
```

```
    }
}
```

Reasoning to interviewer:

- Streaming keeps heap usage predictable and lets me short-circuit once I have enough data for the first screen.
- Jackson’s streaming API is faster and more memory-efficient than binding to POJOs for very large payloads.
- Timeouts and buffered reads avoid stalls and reduce syscalls.

Future scope:

- Backpressure-aware UI: progressively render as items arrive.
- Decompress on the fly when servers return Content-Encoding: gzip.
- Switch to pagination or server-side cursors if the feed is unbounded.

Scenario 22: Mutual TLS (client-auth) for a banking client

The server must verify our client identity using a client certificate. We'll present a keypair from a keystore and also trust the bank's server certificate using a truststore.

Answer:

• I'll configure a keystore (client private key + cert chain) via javax.net.ssl.keyStore and a truststore (server CA roots) via javax.net.ssl.trustStore.
• With those set, HttpsURLConnection (or a pooled client) will perform mutual TLS automatically.
• For finer control, I can build an SSLContext programmatically and set a custom SSLSocketFactory.

Code:

// Quick start using system properties (PKCS12 recommended)

```
import javax.net.ssl.HttpsURLConnection;
import java.net.URL;

public class SSLClientAuth {
    public static void main(String[] args) {
        try {
            // Client identity
            System.setProperty("javax.net.ssl.keyStore", "client-keystore.p12");
            System.setProperty("javax.net.ssl.keyStorePassword", "changeit");
            System.setProperty("javax.net.ssl.keyStoreType", "PKCS12");

            // Server trust anchors (bank CA or corporate CA)
            System.setProperty("javax.net.ssl.trustStore", "bank-truststore.jks");
            System.setProperty("javax.net.ssl.trustStorePassword", "changeit");

            URL url = new URL("https://secure.bankserver.com/account");
            HttpsURLConnection conn = (HttpsURLConnection) url.openConnection();
            conn.setConnectTimeout(4000);
            conn.setReadTimeout(10000);
            System.out.println("Response Code: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

// Advanced: programmatic SSLContext (no system-wide side effects)

```
import javax.net.ssl.*;
import java.io.FileInputStream;
import java.net.URL;
import java.security.KeyStore;

public class SSLClientAuthAdvanced {
    public static void main(String[] args) throws Exception {
        // Load client keys
        KeyStore ks = KeyStore.getInstance("PKCS12");
        ks.load(new FileInputStream("client-keystore.p12"), "changeit".toCharArray());
        KeyManagerFactory kmf =
KeyManagerFactory.getInstance(KeyManagerFactory.getDefaultAlgorithm());
        kmf.init(ks, "changeit".toCharArray());

        // Load trusted CAs
        KeyStore ts = KeyStore.getInstance("JKS");
        ts.load(new FileInputStream("bank-truststore.jks"), "changeit".toCharArray());
        TrustManagerFactory tmf =
TrustManagerFactory.getInstance(TrustManagerFactory.getDefaultAlgorithm());
        tmf.init(ts);

        SSLContext ctx = SSLContext.getInstance("TLS");
        ctx.init(kmf.getKeyManagers(), tmf.getTrustManagers(), null);

        HttpsURLConnection conn = (HttpsURLConnection) new
URL("https://secure.bankserver.com").openConnection();
        conn.setSSLSocketFactory(ctx.getSocketFactory());
        System.out.println("Response Code: " + conn.getResponseCode());
    }
}
```

Reasoning to interviewer:

- Separating keystore (who we are) from truststore (who we trust) avoids security confusion.
- Programmatic SSLContext is cleaner in apps that talk to multiple banks with different CA sets.
- PKCS#12 is portable and standard for client keys.

Future scope:

- Pin the server certificate/SPKI for higher assurance.
- Load keys from an HSM or cloud KMS; rotate certs automatically.
- Enforce TLS versions/ciphers via SSLParameters to meet compliance.

Scenario 23: Restrict an admin server to trusted IPs

An internal admin listener should accept connections only from the corporate LAN. Simple whitelist checks at the TCP layer drastically reduce exposure.

Answer:

• On each accept(), I'll read the remote IP and compare it against an allowlist or CIDR ranges.
• If behind a reverse proxy, I'll verify the proxy's IP and then use a forwarded header (carefully) for the original client.
• Deny by default, log attempts, and close the socket immediately.

Code:

```
import java.net.*;
import java.util.List;

public class IPFilterServer {
    // Exact IPs and CIDR blocks allowed
    private static final List<String> ALLOW = List.of(
        "192.168.1.100",
        "192.168.1.105",
        "10.0.0.0/24"   // whole LAN range
    );

    public static void main(String[] args) {
        try (ServerSocket server = new ServerSocket(8080)) {
            System.out.println("Admin server on :8080");
            while (true) {
                Socket client = server.accept();
                String ip = client.getInetAddress().getHostAddress();

                if (isAllowed(ip)) {
                    System.out.println("Accepted " + ip);
                    // handle admin request here...
                    client.close(); // demo: close after accept
                } else {
                    System.out.println("Rejected " + ip);
                    client.close();
                }
            }
        } catch (Exception e) { e.printStackTrace(); }
    }

    private static boolean isAllowed(String ip) {
        for (String rule : ALLOW) {
            if (rule.contains("/")) {
                if (cidrMatch(ip, rule)) return true;
            } else if (rule.equals(ip)) {
                return true;
```

```
            }
        }
        return false;
    }

    // Simple IPv4 CIDR match: ip like "10.0.0.12", cidr "10.0.0.0/24"
    private static boolean cidrMatch(String ip, String cidr) {
        try {
            String[] parts = cidr.split("/");
            int prefix = Integer.parseInt(parts[1]);
            int mask = prefix == 0 ? 0 : ~((1 << (32 - prefix)) - 1);

            int ipInt = ipv4ToInt(ip);
            int netInt = ipv4ToInt(parts[0]);
            return (ipInt & mask) == (netInt & mask);
        } catch (Exception e) {
            return false;
        }
    }

    private static int ipv4ToInt(String ip) {
        String[] a = ip.split("\\.");
        return (Integer.parseInt(a[0]) << 24)
            | (Integer.parseInt(a[1]) << 16)
            | (Integer.parseInt(a[2]) << 8)
            |  Integer.parseInt(a[3]);
    }
}
```

Reasoning to interviewer:

- Doing the check at accept-time is cheap and effective—bad clients never reach app logic.
- Supporting CIDR avoids maintaining long lists of individual IPs.
- If a reverse proxy is used, I'd only trust X-Forwarded-For from that proxy's IP and still enforce an internal allowlist.

Future scope:

- Swap the in-memory allowlist for a signed config (DB or file with reloading).
- Add mTLS on the admin port so IP filtering isn't the only control.
- Rate-limit and alert on repeated rejects to detect scanning.

Scenario 24: Adding API keys via custom HTTP headers

Many APIs require authentication via a custom header (like Authorization: Bearer <token> or X-API-Key). Without this, requests are rejected with 401 Unauthorized.

Answer:

• With HttpURLConnection, I'll use setRequestProperty() to attach the header before making the request.
• This works for standard headers (Authorization, Content-Type) and custom vendor-specific headers.
• The header is part of the request metadata and will be sent automatically with the HTTP request.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class CustomHeaderExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://api.example.com/data");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");

            // Add API key header
            conn.setRequestProperty("Authorization", "Bearer your_api_key_here");

            System.out.println("Response Code: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Custom headers are the standard way to send tokens, API keys, or request metadata.
• setRequestProperty() must be called before getInputStream() or getResponseCode()—otherwise the request may already be sent.
• Using HTTPS is critical so the API key isn't exposed in plaintext.

Future scope:

• Store keys securely in environment variables or a secrets manager.

• Use token refresh (OAuth2) instead of static keys.

• Switch to an HTTP client library with interceptor support (like OkHttp) for automatic header injection

Scenario 25: Detecting hostname → IP changes (dynamic DNS)

Some domains use dynamic DNS or CDNs, where the mapped IP changes. To know if the backend has moved, resolve the hostname periodically and compare with the last known IP.

Answer:

• I'll call InetAddress.getByName(hostname).getHostAddress() to resolve the current IP.
• Compare with the stored IP from the last check. If different, the host's resolved IP has changed.
• This can be used for DNS monitoring, failover logic, or debugging.

Code:

```
import java.net.InetAddress;

public class IPChangeCheck {
    public static void main(String[] args) {
        String hostname = "www.example.com";
        String lastKnownIP = "192.168.1.50";

        try {
            String currentIP = InetAddress.getByName(hostname).getHostAddress();
            if (!currentIP.equals(lastKnownIP)) {
                System.out.println("IP changed from " + lastKnownIP + " to " +
currentIP);
            } else {
                System.out.println("IP unchanged: " + currentIP);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• DNS results can change due to load balancing, failover, or dynamic IP assignment.
• This method only shows the resolved IP at the time of lookup, not historical DNS mapping.
• For full monitoring, you'd query repeatedly and keep logs.

Future scope:

- Handle multiple IPs (round-robin DNS) with InetAddress.getAllByName().
- Cache results for diffing over time.
- Use a scheduled task or monitoring system to alert on changes.

Scenario 26: Reusing HTTP connections with Keep-Alive

Opening a new TCP connection for every request is expensive (TCP + TLS handshake). Persistent connections reuse the same socket for multiple requests, reducing latency.

Answer:

• By default, modern JDKs already send Connection: keep-alive for HTTP/1.1.
• I can explicitly set setRequestProperty("Connection", "keep-alive") to ensure persistence.
• The server must also support keep-alive; otherwise, it closes the connection after the response.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class KeepAliveExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://example.com/resource");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setRequestProperty("Connection", "keep-alive");

            System.out.println("Response Code: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Persistent connections reduce latency in apps that call the same server often.
• HttpURLConnection has a default keep-alive cache (5 connections per host).
• Explicitly setting the header makes intent clear and prevents proxies from downgrading to close.

Future scope:

• Tune keep-alive cache size using http.maxConnections.

• Use HttpClient (Java 11+) with connection pooling for cleaner reuse.

• Monitor idle connections and close them gracefully to avoid resource leaks.

Scenario 27: Configuring timeouts for TCP sockets

In network apps, you don’t want to hang indefinitely if the server is slow or unreachable. Timeouts prevent wasted threads and improve fault tolerance.

Answer:

• Use socket.connect(remote, timeout) to enforce a connection timeout.
• Use socket.setSoTimeout(ms) to enforce read timeouts on blocking read calls.
• This way, both handshake and data waits are bounded.

Code:

```
import java.net.InetSocketAddress;
import java.net.Socket;

public class TimeoutExample {
    public static void main(String[] args) {
        try (Socket socket = new Socket()) {
            // 5-second connection timeout
            socket.connect(new InetSocketAddress("example.com", 8080), 5000);

            // 3-second read timeout
            socket.setSoTimeout(3000);

            System.out.println("Connected with timeouts configured.");
        } catch (Exception e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

• Connection timeout avoids indefinite hangs if the server is down or firewall blocks packets.
• Read timeout ensures the app doesn’t freeze waiting for data from a slow or dead server.
• These are critical for resilient systems like trading clients or banking apps.

Future scope:

• Use exponential backoff retry strategies when timeouts happen.
• Configure write timeouts for large uploads (via NIO or frameworks).
• Switch to HttpClient (Java 11+) or Netty for advanced timeout controls.

Scenario 28: Restricting server access to trusted IPs

Imagine you're running an internal **admin server** for a company. This server exposes sensitive operations like managing employee data, triggering builds, or restarting services. You don't want everyone on the internet to access it — only machines inside the corporate LAN (for example, 192.168.1.x) should be allowed.

In such cases, you implement **IP filtering** at the application level. When a client connects, the server checks whether the incoming IP is part of the trusted list. If yes, the request is accepted; if not, the connection is dropped immediately. This adds a simple yet effective **first line of defense**, even before authentication.

Answer:

Maintain a whitelist of trusted IPs. For every new connection, fetch the client's IP address and compare it against the whitelist. If the client isn't trusted, close the socket right away.

Code:

```
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Arrays;
import java.util.List;

public class IPFilterServer {
    private static final List<String> allowedIPs = Arrays.asList(
        "192.168.1.100",
        "192.168.1.105"
    );

    public static void main(String[] args) {
        try (ServerSocket server = new ServerSocket(8080)) {
            System.out.println("Server started on port 8080");

            while (true) {
                Socket client = server.accept();
                String clientIP = client.getInetAddress().getHostAddress();

                if (allowedIPs.contains(clientIP)) {
                    System.out.println("Accepted connection from " + clientIP);
                    // process the request here
                } else {
                    System.out.println("Rejected connection from " + clientIP);
                    client.close();
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
```

```
    }
}
```

Reasoning to interviewer:

- This ensures only trusted corporate IPs can talk to the server.
- Helps reduce attack surface even before authentication/authorization.
- Easy to implement with minimal overhead since IP checking is done at socket accept.
- Prevents unauthorized access if someone accidentally exposes the server to the internet.

Future scope:

- Instead of a hardcoded list, load allowed IPs from config or database.
- Support CIDR ranges (e.g., 192.168.1.0/24) for whole subnets.
- Add logging/auditing for rejected connection attempts.
- Combine with TLS client certificates for stronger identity checks.

Scenario 29: Adding API keys via custom HTTP headers

Some APIs don't allow query-string tokens and instead expect authentication in the header (like Authorization: Bearer <token> or X-API-Key: <key>). Without it, you'll get a 401 Unauthorized.

Answer:

• In HttpURLConnection, I'll call setRequestProperty() before sending the request to add the header.
• Works for both standard and custom headers.
• Always use HTTPS so the API key isn't leaked in plaintext.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class CustomHeaderExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://api.example.com/data");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");

            // Add API key in the header
            conn.setRequestProperty("Authorization", "Bearer your_api_key_here");

            System.out.println("Response Code: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Adding headers via setRequestProperty() is the cleanest way to attach metadata.
• This keeps API keys out of URLs (which may get logged by proxies).
• Always set the header before calling getResponseCode() or getInputStream(), since those trigger the actual request.

Future scope:

• Refresh tokens automatically if expired (OAuth2).

• Centralize key injection using an HTTP client wrapper.

• Store keys in environment variables or a secret vault, not code.

Scenario 30: Detecting hostname → IP changes (dynamic DNS)

If your service connects to www.example.com, that DNS name might resolve to different IPs at different times—due to CDN routing or failover. You may want to detect this change for monitoring or reconnection logic.

Answer:

• Use InetAddress.getByName(hostname) to resolve the current IP.
• Compare it against a stored value. If different, the DNS mapping has changed.
• This is useful for DNS-aware clients or for failover detection.

Code:

```
import java.net.InetAddress;

public class IPChangeCheck {
    public static void main(String[] args) {
        String hostname = "www.example.com";
        String lastKnownIP = "192.168.1.50"; // previously stored

        try {
            String currentIP = InetAddress.getByName(hostname).getHostAddress();
            if (!currentIP.equals(lastKnownIP)) {
                System.out.println("IP changed from " + lastKnownIP + " to " +
currentIP);
            } else {
                System.out.println("IP unchanged: " + currentIP);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• DNS results can rotate (round-robin) or shift (failover).
• This check only confirms the *resolved* IP, not the authoritative DNS mapping history.
• To handle multiple IPs, I'd use InetAddress.getAllByName().

Future scope:

• Periodically resolve and log changes for monitoring.
• Use TTL-aware caching instead of hard-coding the last IP.
• Integrate alerts if a hostname suddenly points to an unexpected IP range.

Scenario 31: Reusing TCP connections with Keep-Alive

HTTP/1.1 defaults to persistent connections, but sometimes proxies or older servers may close them unless explicitly told.

Answer:

• I'll set the Connection: keep-alive header to request persistence.
• Java's HttpURLConnection already reuses TCP connections under the hood, but explicitly setting the header ensures both ends agree.
• This saves on expensive handshakes (TCP and TLS).

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class KeepAliveExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://example.com/resource");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setRequestProperty("Connection", "keep-alive");

            System.out.println("Response Code: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Keep-alive lets multiple requests share the same socket, reducing latency.
• By default, JDK maintains a keep-alive cache of 5 connections per host.
• If the server ignores keep-alive, the client automatically falls back to new connections.

Future scope:

• Tune keep-alive pool size with system property http.maxConnections.
• Use HttpClient (Java 11+) or OkHttp for more efficient connection pooling.
• Monitor connection reuse to avoid "connection storms" under load.

Scenario 32: Configuring timeouts for a TCP client

In a persistent client-server system, you never want to hang forever on a bad network. For example, if a stock trading service is slow, the client must time out quickly so it can retry or fail gracefully.

Answer:

• Use Socket.connect(remote, timeout) to define a maximum time allowed to establish the connection.
• Use socket.setSoTimeout(ms) to apply a read timeout so blocking reads throw an exception if no data arrives.
• This way, both connection setup and data retrieval are bounded.

Code:

```
import java.net.InetSocketAddress;
import java.net.Socket;

public class TimeoutExample {
    public static void main(String[] args) {
        try (Socket socket = new Socket()) {
            // 5-second connection timeout
            socket.connect(new InetSocketAddress("example.com", 8080), 5000);

            // 3-second read timeout
            socket.setSoTimeout(3000);

            System.out.println("Connected with timeouts configured.");
        } catch (Exception e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

• Connection timeout protects against unreachable hosts or firewalls silently dropping SYN packets.
• Read timeout prevents threads from hanging when a server stalls mid-transaction.
• These are critical for high-availability apps like trading or chat clients.

Future scope:

• Add retries with exponential backoff.
• Also configure write timeouts (via NIO or higher-level clients).
• Monitor and log timeout events for troubleshooting.

Scenario 33: Enforcing HTTPS with certificate validation

For a banking client, it's non-negotiable that all communication is encrypted and verified. Simply trusting plaintext HTTP or invalid certs is a huge risk.

Answer:

• Use an https:// URL so Java creates an HttpsURLConnection.
• During the SSL handshake, Java validates the server's certificate chain against the default trust store (or a custom one).
• If the certificate is expired, self-signed, or issued by an unknown CA, the handshake fails.
• Optionally, use a custom TrustManager for stricter policies (like certificate pinning).

Code:

```
import javax.net.ssl.HttpsURLConnection;
import java.net.URL;

public class HttpsConnectionExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("https://secure.bankserver.com/data");
            HttpsURLConnection conn = (HttpsURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setConnectTimeout(4000);
            conn.setReadTimeout(10000);

            System.out.println("Response Code: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Java's default trust store contains widely trusted CAs, which covers most production servers.
• This ensures man-in-the-middle attacks are blocked if the cert isn't valid.
• For banking-level security, often mutual TLS is used (server validates client too).

Future scope:

• Implement certificate pinning to ensure only known server certs are trusted.
• Rotate trust stores regularly to stay updated with root CAs.
• Upgrade TLS versions and enforce strong cipher suites via SSLParameters.

Scenario 34: Sending lightweight heartbeats with UDP

For monitoring or IoT-style heartbeats, maintaining a TCP connection is overkill. UDP is faster and cheaper for simple one-shot messages like “I’m alive.”

Answer:

• Create a DatagramSocket for sending packets.
• Prepare a DatagramPacket with the message bytes, server address, and port.
• Call send(packet)—no connection setup is needed.

Code:

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;

public class UDPClient {
    public static void main(String[] args) {
        try (DatagramSocket socket = new DatagramSocket()) {
            String msg = "Hello UDP Server!";
            byte[] buffer = msg.getBytes();

            InetAddress serverAddr = InetAddress.getByName("localhost");
            DatagramPacket packet =
                new DatagramPacket(buffer, buffer.length, serverAddr, 9876);

            socket.send(packet);
            System.out.println("Message sent to UDP server.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• UDP is connectionless—no handshake, just fire-and-forget. That makes it perfect for heartbeats.

• The downside is no guarantee of delivery, so it’s best for signals that can tolerate loss.

• For banking-style systems, I’d use TCP/TLS instead; for monitoring, UDP is ideal.

Future scope:

• Add sequence numbers so the server can detect missed/lost heartbeats.

• Encrypt messages if confidentiality is required.

• Switch to multicast UDP if many monitoring servers need the same heartbeat.

Scenario 35: Verify downloaded file type before saving

You don’t want to save an HTML error page as “.pdf”. Before writing bytes to disk, first confirm the response is a PDF/image by checking the Content-Type header.

Answer:

• After opening the connection, inspect getContentType() (e.g., application/pdf, image/png).
• Optionally, double-check with getHeaderField("Content-Type") (some servers add charset).
• If the type isn’t one you expect, abort or route it to an error handler.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class ContentTypeCheck {
    public static void main(String[] args) {
        try {
            URL url = new URL("https://example.com/document.pdf");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setConnectTimeout(4000);
            conn.setReadTimeout(15000);

            int code = conn.getResponseCode();
            if (code != HttpURLConnection.HTTP_OK) {
                System.out.println("Unexpected status: " + code);
                return;
            }

            String contentType = conn.getContentType(); // same as header "Content-
Type"
            System.out.println("Content-Type: " + contentType);

            if (contentType != null &&
                (contentType.startsWith("application/pdf")
                 || contentType.startsWith("image/"))) {
                System.out.println("Type looks good — proceed to save stream to file.");
                // ... copy conn.getInputStream() to file
            } else {
                System.out.println("Unexpected type. Not saving.");
                // ... maybe read error body for diagnostics
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
```

```
    }
}
```

Reasoning to interviewer:

• Relying on headers prevents corrupt UI flows (e.g., saving an HTML error page).

• Whitelisting media types reduces surprises; MIME is more reliable than file extensions.

• I still validate bytes server-side (e.g., magic numbers) if security is critical.

Future scope:

• Inspect magic bytes (PDF starts with “%PDF”, PNG starts with \x89PNG) for stronger validation.

• Enforce maximum size limits before saving to disk.

• Use streaming copy with checksums (e.g., SHA-256) to verify integrity.

Scenario 36: Fetch only if the news changed (conditional GET)

Polling an endpoint hourly shouldn't re-download the same payload. Use HTTP caching headers: If-Modified-Since and, even better, If-None-Match with ETags.

Answer:

• Send If-Modified-Since with the last known Last-Modified timestamp to ask, "only send if newer."
• Prefer ETags: send If-None-Match with the last ETag; a 304 Not Modified means you can reuse cached data.
• On a 200 OK, update your stored Last-Modified/ETag for next time.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class ConditionalGetExample {
    // Persist these between runs (DB/file). Using in-memory for demo:
    private static volatile long lastModified = 0L;
    private static volatile String lastEtag = null;

    public static void main(String[] args) {
        try {
            URL url = new URL("https://news.example.com/latest");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setConnectTimeout(3000);
            conn.setReadTimeout(10000);

            if (lastModified > 0) conn.setIfModifiedSince(lastModified);
            if (lastEtag != null) conn.setRequestProperty("If-None-Match", lastEtag);

            int code = conn.getResponseCode();
            if (code == HttpURLConnection.HTTP_NOT_MODIFIED) {
                System.out.println("No new updates (304). Use cached content.");
                return;
            }

            if (code == HttpURLConnection.HTTP_OK) {
                System.out.println("Update available. Downloading...");
                // ... read and process JSON body here (streaming if large)

                // Capture validators for next request
                lastModified = conn.getLastModified(); // from "Last-Modified"
                lastEtag = conn.getHeaderField("ETag");
                System.out.println("Saved validators -> Last-Modified: " + lastModified +
", ETag: " + lastEtag);
            } else {
```

```
                System.out.println("Unexpected response: " + code);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- ETags are more precise than timestamps across CDNs and clock skews.
- This pattern cuts bandwidth and speeds up perceived freshness checks.
- Storing validators is trivial and pays off immediately in production.

Future scope:

- Back off polling frequency when content is quiet; increase under heavy news.
- Cache body on disk with validators for instant app startup.
- Move to HTTP/2 or push subscriptions if the API supports server-initiated updates.

Scenario 37: Talking to a legacy/custom TCP protocol

No HTTP here—just raw TCP with a binary or text framing you define. Think of a trading gateway or device firmware talking over a custom wire format.

Answer:

• Use Socket and work with InputStream/OutputStream directly.
• Define a clear frame: e.g., [length (int)][opcode (byte)][payload…].
• Use DataInputStream/DataOutputStream for fixed-width primitives; set timeouts to avoid hangs.

Code:

```
import java.io.*;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class CustomProtocolClient {
    public static void main(String[] args) {
        String ip = "192.168.1.100";
        int port = 5555;

        try (Socket socket = new Socket()) {
            socket.connect(new InetSocketAddress(ip, port), 4000);
            socket.setSoTimeout(4000);

            try (DataOutputStream out = new DataOutputStream(new
BufferedOutputStream(socket.getOutputStream()));
                 DataInputStream  in  = new DataInputStream(new
BufferedInputStream(socket.getInputStream()))) {

                // Example frame: [len:int][opcode:byte][payload:bytes]
                byte opcode = 0x01; // HELLO
                byte[] payload =
"HELLO_SERVER".getBytes(StandardCharsets.UTF_8);
                int len = 1 + payload.length;

                out.writeInt(len);
                out.writeByte(opcode);
                out.write(payload);
                out.flush();

                // Read response frame
                int respLen = in.readInt();
                byte respCode = in.readByte();
                byte[] resp = in.readNBytes(respLen - 1);
```

```
            System.out.println("Resp opcode=" + respCode + ", body=" + new
String(resp, StandardCharsets.UTF_8));
          }
       } catch (IOException e) {
          e.printStackTrace();
       }
    }
}
```

Reasoning to interviewer:

- Length-prefixed framing avoids “where does this message end?” problems on TCP streams.
- DataInputStream#readNBytes ensures we read the exact payload length; timeouts protect threads.
- Using opcodes keeps the protocol extensible (new commands without breaking old ones).

Future scope:

- Add message checksums and a version field for negotiation.
- TLS via SSLSocket or run behind a terminating proxy for confidentiality.
- Introduce async/NIO client if concurrency (many sockets) becomes high.

Scenario 38: Checking server reachability by IP and port

In microservice environments, you often need lightweight **health checks**. For example, before sending requests from Service A to Service B, you want to verify that B is actually up and listening on the expected port. If the port is closed or unreachable (due to firewall, crash, or network partition), it's better to fail fast rather than wasting resources retrying API calls.

Answer:

You can try opening a Socket to the given IP and port. If the connection is established successfully, the server is reachable. If not, an exception is thrown (e.g., ConnectException, SocketTimeoutException). This is a simple connectivity probe that can be run before sending actual business requests.

Code:

```
import java.net.InetSocketAddress;
import java.net.Socket;

public class ServerReachabilityCheck {
    public static void main(String[] args) {
        String serverIp = "192.168.1.1";
        int port = 8080;

        try (Socket socket = new Socket()) {
            // Set a small timeout for faster health checks
            socket.connect(new InetSocketAddress(serverIp, port), 3000);
            System.out.println("Server is reachable at " + serverIp + ":" + port);
        } catch (Exception e) {
            System.out.println("Cannot reach server: " + e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- A plain Socket connection test is enough to confirm if the server is listening.
- Adding a timeout (e.g., 3 seconds) avoids long hangs when the host is down or firewalled.
- This approach is protocol-agnostic — works for HTTP, gRPC, or any service running on TCP.
- Useful in monitoring, load balancers, and custom readiness checks.

Future scope:

- Add retries with exponential backoff for transient failures.
- Log metrics (success/failure rate, latency) for observability.
- Extend to layer-7 checks (e.g., HTTP GET /health endpoint) for more detailed health validation.
- Integrate with service discovery or orchestrators (Kubernetes, Consul) for automated failover.

Scenario 39: Simple client-server time service

This is the classic “Hello world” of socket programming. A server listens on a port; a client connects, sends a request (or just connects), and the server responds with the current time.

Answer:

• On the server side, use ServerSocket to bind to a port, accept a connection, and write the response.
• On the client side, use Socket to connect, and read the response using an input stream.
• This pattern works on the same machine (localhost) or across a LAN.

Code (Server):

```
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Date;

public class SimpleServer {
    public static void main(String[] args) {
        try (ServerSocket server = new ServerSocket(9090)) {
            System.out.println("Server started on port 9090");

            while (true) {
                Socket client = server.accept();
                try (PrintWriter out = new PrintWriter(client.getOutputStream(), true)) {
                    out.println("Current Date & Time: " + new Date());
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Code (Client):

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.Socket;

public class SimpleClient {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 9090);
```

```
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()))) {

            String response = in.readLine();
            System.out.println("Server says: " + response);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- ServerSocket is the entry point for TCP servers in Java. It blocks on accept() until a client arrives.
- Keeping the server in a loop allows multiple sequential client connections.
- Wrapping the streams (PrintWriter, BufferedReader) makes text-based protocols easy.

Future scope:

- For concurrency, handle each client in a separate thread or thread pool.
- Add request parsing so the client can ask for “date” vs. “time” specifically.
- Secure the channel with SSLServerSocket if used outside a trusted LAN.

Scenario 40: Calling a weather API with API key authentication

Most modern APIs require authentication—either via OAuth2 tokens (Authorization: Bearer …) or custom headers like X-API-Key.

Answer:

• With HttpURLConnection, call setRequestProperty() to attach the required header before sending the request.
• Always use https:// endpoints so the key isn't exposed in plaintext.
• If the API expects a different header (X-API-Key), adjust accordingly.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class ApiKeyClient {
    public static void main(String[] args) {
        try {
            URL url = new URL("https://weatherapi.example.com/data");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");

            // Add API key in Authorization header (or use X-API-Key if required)
            conn.setRequestProperty("Authorization", "Bearer my_api_key_123");

            int code = conn.getResponseCode();
            System.out.println("Response Code: " + code);
            // Optionally, read the JSON body with InputStreamReader
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• API keys are best passed in headers (not query params), since headers aren't usually logged by proxies.
• setRequestProperty() ensures the header is included before the request is executed.
• For repeated calls, wrap this in a utility method or switch to Java's newer HttpClient for cleaner code.

Future scope:

• Rotate API keys securely, don’t hardcode them. Use env variables or secret managers.

• If the API supports OAuth2, implement token refresh.

• Add retries, backoff, and error handling (e.g., for 401 Unauthorized).

Scenario 41: Mutual TLS (client cert) to a secure enterprise server

Enterprise backends often require the client to prove its identity with a certificate, not just trust the server. So the client must present a certificate + private key (keystore) and also trust the server CA (truststore).

Answer:

• Put the client keypair + cert chain in a keystore (PKCS12 is common), and the server's CA(s) in a truststore.
• Either use system properties (quick setup) or build an SSLContext programmatically (cleaner, no JVM-global side effects).
• When the TLS handshake runs, Java presents the client cert and validates the server's cert against your truststore.

Code:

// Option A — Quick: system properties (simple demos / single-tenant apps)

```
import javax.net.ssl.HttpsURLConnection;
import java.net.URL;

public class SSLClientAuth {
    public static void main(String[] args) throws Exception {
        System.setProperty("javax.net.ssl.keyStore", "client-keystore.p12");
        System.setProperty("javax.net.ssl.keyStorePassword", "changeit");
        System.setProperty("javax.net.ssl.keyStoreType", "PKCS12");

        System.setProperty("javax.net.ssl.trustStore", "enterprise-truststore.jks");
        System.setProperty("javax.net.ssl.trustStorePassword", "changeit");

        URL url = new URL("https://secure.enterprise.com/data");
        HttpsURLConnection conn = (HttpsURLConnection) url.openConnection();
        conn.setConnectTimeout(4000);
        conn.setReadTimeout(10000);
        System.out.println("Response Code: " + conn.getResponseCode());
    }
}
```

// Option B — Preferred: programmatic SSLContext (multiple endpoints / better isolation)

```
import javax.net.ssl.*;
import java.io.FileInputStream;
import java.net.URL;
import java.security.KeyStore;

public class SSLClientAuthCtx {
    public static void main(String[] args) throws Exception {
```

```
        KeyStore ks = KeyStore.getInstance("PKCS12");
        ks.load(new FileInputStream("client-keystore.p12"), "changeit".toCharArray());
        KeyManagerFactory kmf =
KeyManagerFactory.getInstance(KeyManagerFactory.getDefaultAlgorithm());
        kmf.init(ks, "changeit".toCharArray());

        KeyStore ts = KeyStore.getInstance("JKS");
        ts.load(new FileInputStream("enterprise-truststore.jks"),
"changeit".toCharArray());
        TrustManagerFactory tmf =
TrustManagerFactory.getInstance(TrustManagerFactory.getDefaultAlgorithm());
        tmf.init(ts);

        SSLContext ctx = SSLContext.getInstance("TLS");
        ctx.init(kmf.getKeyManagers(), tmf.getTrustManagers(), null);

        HttpsURLConnection conn = (HttpsURLConnection) new
URL("https://secure.enterprise.com/data").openConnection();
        conn.setSSLSocketFactory(ctx.getSocketFactory());
        System.out.println("Response Code: " + conn.getResponseCode());
    }
}
```

Reasoning to interviewer:

- Keystore = “who I am”; truststore = “who I trust.” Keeping them separate avoids accidental trust of the wrong CAs.
- Programmatic SSLContext is safer when talking to multiple partners (different CAs) in the same JVM.
- PKCS#12 is portable for client keys; JKS still common for truststores.

Future scope:

- Certificate pinning (SPKI) for stronger server validation.
- Load keys from HSM/KMS; automate rotation.
- Enforce TLS versions/ciphers via SSLParameters and add OCSP/CRL checks if required by policy.

Scenario 42: Download only if the document changed (conditional GET)

Repeatedly syncing the same file wastes bandwidth. Use validators to ask the server: “send only if newer.”

Answer:

• Send If-Modified-Since (timestamp) and/or If-None-Match (ETag).
• On 304 Not Modified, reuse your cached copy. On 200 OK, update your cached file and store new validators.

Code:

```
import java.io.*;
import java.net.HttpURLConnection;
import java.net.URL;

public class ConditionalGetClient {
    // Persist these across runs (e.g., a small file/db)
    private static volatile long lastModified = 0L;
    private static volatile String etag = null;

    public static void main(String[] args) throws Exception {
        URL url = new URL("https://files.example.com/document");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        if (lastModified > 0) conn.setIfModifiedSince(lastModified);
        if (etag != null) conn.setRequestProperty("If-None-Match", etag);

        int code = conn.getResponseCode();
        if (code == HttpURLConnection.HTTP_NOT_MODIFIED) {
            System.out.println("No new updates (304).");
            return;
        }
        if (code == HttpURLConnection.HTTP_OK) {
            try (InputStream in = conn.getInputStream();
                 OutputStream out = new FileOutputStream("document.bin")) {
                in.transferTo(out); // Java 9+
            }
            lastModified = conn.getLastModified();          // from "Last-Modified"
            etag = conn.getHeaderField("ETag");              // strong/weak ETag
            System.out.println("Downloaded updated file. Saved validators.");
        } else {
            System.out.println("Unexpected response: " + code);
        }
    }
}
```

Reasoning to interviewer:

- ETags are more precise than timestamps (no clock skew issues), so I prefer If-None-Match when available.
- Storing validators is tiny effort for big bandwidth wins.
- Works nicely with CDNs that honor cache validators.

Future scope:

- Add gzip (Accept-Encoding: gzip) and stream to disk to reduce memory.
- Backoff schedule when there are repeated 304s.
- If the API supports ranges, combine with resume downloads on flaky networks.

Scenario 43: Reuse HTTP/TCP connections with Keep-Alive

Frequent calls to the same host shouldn't pay the cost of new TCP+TLS handshakes every time.

Answer:

• HTTP/1.1 keeps connections alive by default, but I'll still set Connection: keep-alive and avoid disconnect() to keep the socket reusable.
• Make sure to fully read/close each response body—otherwise the connection won't return to the pool.

Code:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

public class KeepAliveExample {
    public static void main(String[] args) throws Exception {
        URL url = new URL("https://example.com/api/data");
        for (int i = 0; i < 3; i++) {
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setRequestProperty("Connection", "keep-alive");
            conn.setConnectTimeout(3000);
            conn.setReadTimeout(8000);

            int code = conn.getResponseCode();
            System.out.println("Request " + i + " -> " + code);

            // Read the body so the connection can be reused
            try (BufferedReader br = new BufferedReader(new
InputStreamReader(conn.getInputStream()))) {
                while (br.readLine() != null) { /* drain */ }
            }
            // Do NOT call conn.disconnect(); let the implementation reuse the socket
        }
    }
}
```

Reasoning to interviewer:

• Connection reuse collapses latency (no SYN/ACK, no TLS handshake) and reduces CPU.

• Draining/closing the response input stream is essential; otherwise the keep-

alive connection is poisoned.

• For heavy use, Java 11+ HttpClient or OkHttp gives better pooling, HTTP/2 multiplexing, and cleaner APIs.

Future scope:

• Tune pool size (http.maxConnections) and idle timeouts.

• Prefer HTTP/2 (multiplexed streams over one TCP connection).

• Add retries with backoff and circuit breakers around the client.

Scenario 44: Sending JSON in a POST request

When a REST API expects structured data, you must serialize your object into JSON and send it as the request body. Without the proper header, the server may reject or misinterpret the request.

Answer:

- Open an HttpURLConnection to the target endpoint.
- Set request method to POST and header Content-Type: application/json.
- Call setDoOutput(true) to enable request body writing.
- Write the JSON string into the output stream in UTF-8 encoding.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;
import java.io.OutputStream;

public class PostJsonExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://example.com/api/users");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("POST");
            conn.setRequestProperty("Content-Type", "application/json");
            conn.setRequestProperty("Accept", "application/json");
            conn.setDoOutput(true);

            String json = "{\"name\":\"Alice\",\"age\":25}";
            try (OutputStream os = conn.getOutputStream()) {
                os.write(json.getBytes("UTF-8"));
            }

            int code = conn.getResponseCode();
            System.out.println("Response Code: " + code);
            // Optionally read response body here
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Setting the Content-Type header ensures the server parses the payload correctly.

• UTF-8 encoding avoids issues with non-ASCII characters in names or other text.

• Always close the stream; using try-with-resources guarantees cleanup.

Future scope:

• Use a JSON library (Jackson, Gson) to map POJOs → JSON automatically.

• Handle response body for success/error messages.

• Add error handling for status codes (e.g., 400 Bad Request).

Scenario 45: Downloading large files in chunks

For files like videos or ZIP archives, reading them into memory at once is unsafe. Instead, stream data in manageable chunks and flush to disk as you go.

Answer:

• Open the HTTP connection and get an input stream.
• Wrap with BufferedInputStream for efficient reads.
• Create a FileOutputStream and loop over the input, reading 4 KB (or bigger) at a time.
• Write each chunk to disk until EOF.

Code:

```
import java.io.BufferedInputStream;
import java.io.FileOutputStream;
import java.net.HttpURLConnection;
import java.net.URL;

public class LargeFileDownload {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://example.com/largefile.zip");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setConnectTimeout(5000);
            conn.setReadTimeout(60000);

            try (BufferedInputStream in = new
BufferedInputStream(conn.getInputStream());
                 FileOutputStream out = new FileOutputStream("download.zip")) {

                byte[] buffer = new byte[8192]; // 8 KB chunks
                int bytesRead;
                while ((bytesRead = in.read(buffer)) != -1) {
                    out.write(buffer, 0, bytesRead);
                }
            }

            System.out.println("File downloaded successfully.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- Streaming prevents OutOfMemoryError when handling huge files.
- Buffer size (4–8 KB or more) balances memory usage and performance.
- Writing directly to a file ensures constant memory usage no matter the file size.

Future scope:

- Show progress by comparing bytesRead against Content-Length.
- Resume partial downloads using HTTP Range header.
- Add checksum validation (e.g., SHA-256) after download for integrity.

Scenario 46: Performing DNS lookup for a domain

Sometimes your app needs to resolve a domain into IP addresses and host information, for example when troubleshooting connectivity or logging server details.

Answer:

• Use InetAddress.getByName(host) for a single resolution (usually the first IP returned).
• Use InetAddress.getAllByName(host) to list all IPs if the domain maps to multiple servers (like behind a CDN).
• Methods like getHostName() and getCanonicalHostName() help differentiate between aliases and the fully qualified canonical name.

Code:

```
import java.net.InetAddress;

public class DNSLookupExample {
    public static void main(String[] args) {
        try {
            InetAddress address = InetAddress.getByName("www.example.com");
            System.out.println("IP Address: " + address.getHostAddress());
            System.out.println("Host Name: " + address.getHostName());
            System.out.println("Canonical Name: " +
address.getCanonicalHostName());

            System.out.println("\nAll associated addresses:");
            InetAddress[] all = InetAddress.getAllByName("www.example.com");
            for (InetAddress a : all) {
                System.out.println(" - " + a.getHostAddress());
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• DNS lookups may return multiple IPs; using getAllByName() ensures you see them all.
• Canonical hostnames are useful when load balancers or aliases are in play.
• DNS resolution depends on the system's resolver, so caching and TTLs apply.

Future scope:

- Integrate with a custom DNS library if you need direct DNS record types (MX, TXT).
- Cache resolved IPs to avoid repeated lookups in latency-sensitive apps.
- Handle IPv6 gracefully by checking for both A and AAAA records.

Scenario 47: Reverse DNS lookup for an IP address

Reverse DNS lookups can add context in logs, like showing dns.google instead of just 8.8.8.8.

Answer:

• Use InetAddress.getByName(ip) to wrap the IP.
• Then call getHostName() or getCanonicalHostName() to attempt a reverse lookup.
• If no PTR record exists, it just returns the IP string.

Code:

```
import java.net.InetAddress;

public class ReverseDNSExample {
    public static void main(String[] args) {
        try {
            InetAddress address = InetAddress.getByName("8.8.8.8");
            System.out.println("IP Address: " + address.getHostAddress());
            System.out.println("Host Name: " + address.getHostName());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Reverse lookups rely on PTR records in DNS; many IPs don’t have them.
• This is useful for logging, auditing, or access control, but not for security decisions (reverse DNS can be spoofed).
• Always keep the original IP alongside the resolved name in logs.

Future scope:

• Use both hostname and IP in security logs for clarity.
• Fall back to IP if reverse resolution fails.
• Cache lookups to reduce DNS load in high-traffic servers.

Scenario 48: Checking if a port is available

Before starting a server (say on port 8080), you need to confirm it's not already bound by another process.

Answer:

• Attempt to create a ServerSocket on the desired port.
• If it succeeds, the port is free. If it throws an exception, the port is already in use.
• Always close the socket immediately after testing.

Code:

```
import java.net.ServerSocket;

public class PortCheckExample {
    public static void main(String[] args) {
        int port = 8080;
        try (ServerSocket socket = new ServerSocket(port)) {
            System.out.println("Port " + port + " is available.");
        } catch (Exception e) {
            System.out.println("Port " + port + " is already in use.");
        }
    }
}
```

Reasoning to interviewer:

• Binding tests if the OS allows exclusive use of that port.
• Must close the test socket, otherwise you'll block yourself from binding later.
• This method checks availability at the moment—it doesn't guarantee the port will still be free later.

Future scope:

• Handle privileged ports (<1024) where admin rights are required.
• Build retry logic with alternative ports if the preferred one is unavailable.
• Monitor and log port usage for operational diagnostics.

Scenario 49: Encoding and decoding URL parameters

When building a search client, user queries may contain spaces, punctuation, or symbols (+, ?, &) that break a URL if not encoded.

Answer:

• Use URLEncoder.encode() to safely encode parameters into URL-safe form.
• Use URLDecoder.decode() to convert them back.
• Always specify UTF-8 as the charset for consistency across platforms.

Code:

```
import java.net.URLEncoder;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;

public class URLEncodeDecodeExample {
    public static void main(String[] args) {
        try {
            String query = "Java Networking Basics!";
            String encoded = URLEncoder.encode(query,
StandardCharsets.UTF_8.toString());
            String decoded = URLDecoder.decode(encoded,
StandardCharsets.UTF_8.toString());

            System.out.println("Original: " + query);
            System.out.println("Encoded: " + encoded);
            System.out.println("Decoded: " + decoded);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• URLEncoder is critical for constructing safe query strings—otherwise spaces or & break the URL.
• Encoding is different from escaping HTML—it's purely for URL transport.
• Always decode with the same charset to avoid mismatches.

Future scope:

• Use libraries like URIBuilder (Apache HttpClient) to manage parameters more cleanly.
• Validate user input before encoding to prevent injection attacks.

Scenario 50: Maintaining cookies across requests

Many servers rely on cookies (like JSESSIONID) for login sessions. If the client doesn't send cookies, subsequent requests won't be authenticated.

Answer:

• Use a CookieManager and set it as the default with CookieHandler.setDefault().
• Once set, cookies returned by the server are automatically stored and resent in later requests.
• This avoids manually parsing Set-Cookie headers.

Code:

```
import java.net.CookieHandler;
import java.net.CookieManager;
import java.net.HttpURLConnection;
import java.net.URL;

public class CookieExample {
    public static void main(String[] args) {
        try {
            CookieManager manager = new CookieManager();
            CookieHandler.setDefault(manager);

            // First request: login
            URL url = new URL("http://example.com/login");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            System.out.println("First Response: " + conn.getResponseCode());

            // Second request: cookies auto-included
            URL url2 = new URL("http://example.com/dashboard");
            HttpURLConnection conn2 = (HttpURLConnection) url2.openConnection();
            System.out.println("Second Response: " + conn2.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• CookieManager acts as a store for session cookies—exactly how browsers keep you logged in.

• With this in place, you don't need to manually manage headers like Cookie

and Set-Cookie.

- Important for APIs requiring login, CSRF tokens, or session persistence.

Future scope:

- Persist cookies across runs with a custom CookieStore.
- Enforce HTTPS-only or HttpOnly flags when handling sensitive cookies.

Scenario 51: Resuming large file downloads with Range requests

If a download fails halfway, restarting from scratch wastes bandwidth. HTTP Range requests let you resume from the last byte you saved.

Answer:

- Use the Range header to request bytes=<offset>-.
- Open the existing file with RandomAccessFile and seek to the last saved position.
- Append incoming bytes from the server to continue the download.

Code:

```
import java.io.RandomAccessFile;
import java.io.InputStream;
import java.net.HttpURLConnection;
import java.net.URL;

public class ResumeDownloadExample {
    public static void main(String[] args) {
        try {
            String fileURL = "http://example.com/largefile.zip";
            String filePath = "download.zip";

            long downloaded = 500000; // already saved bytes

            URL url = new URL(fileURL);
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestProperty("Range", "bytes=" + downloaded + "-");

            try (RandomAccessFile file = new RandomAccessFile(filePath, "rw");
                 InputStream in = conn.getInputStream()) {

                file.seek(downloaded);
                byte[] buffer = new byte[8192]; // 8 KB chunks
                int bytesRead;
                while ((bytesRead = in.read(buffer)) != -1) {
                    file.write(buffer, 0, bytesRead);
                }
            }

            System.out.println("Resumed download complete.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- The Range header makes the server return 206 Partial Content instead of 200 OK.
- Using RandomAccessFile allows precise writes starting at the resume point.
- This is how download managers implement pause/resume.

Future scope:

- Support multi-part parallel downloads by assigning different ranges to different threads.
- Verify integrity with checksums or ETags to ensure resumed content matches.
- Handle servers that don’t support range requests gracefully.

Scenario 52: Uploading a file with multipart/form-data

APIs that handle file uploads (like profile pictures) usually expect multipart/form-data. This format separates fields and files with boundaries so the server knows how to parse them.

Answer:

• Define a boundary string to delimit parts.
• Set the request's Content-Type to multipart/form-data; boundary=<boundary>.
• Write the multipart structure: headers (Content-Disposition, Content-Type), then raw file bytes, then close the boundary.
• Flush and close streams properly so the server knows the request ended.

Code:

```
import java.io.*;
import java.net.HttpURLConnection;
import java.net.URL;

public class MultipartUploadExample {
    public static void main(String[] args) {
        String boundary = "----JavaBoundary" + System.currentTimeMillis();
        String filePath = "profile.jpg";
        File uploadFile = new File(filePath);

        try {
            URL url = new URL("http://example.com/upload");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("POST");
            conn.setDoOutput(true);
            conn.setRequestProperty("Content-Type", "multipart/form-data;
boundary=" + boundary);

            try (OutputStream out = conn.getOutputStream();
                 PrintWriter writer = new PrintWriter(new OutputStreamWriter(out, "UTF-
8"), true);
                 FileInputStream fileIn = new FileInputStream(uploadFile)) {

                // File part
                writer.append("--").append(boundary).append("\r\n");
                writer.append("Content-Disposition: form-data; name=\"file\";
filename=\"")
                      .append(uploadFile.getName()).append("\"\r\n");
                writer.append("Content-Type: image/jpeg\r\n\r\n");
                writer.flush();

                byte[] buffer = new byte[4096];
                int bytesRead;
                while ((bytesRead = fileIn.read(buffer)) != -1) {
```

```
                out.write(buffer, 0, bytesRead);
            }
            out.flush();
            writer.append("\r\n");

            // End boundary
            writer.append("--").append(boundary).append("--").append("\r\n");
            writer.flush();
        }

        System.out.println("Upload Response: " + conn.getResponseCode());
    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

Reasoning to interviewer:

- Multipart encoding is required when mixing files and text fields in one request.
- Boundaries act as separators, so the server knows where file content ends.
- Always close with --boundary-- to signal end of request.

Future scope:

- Add additional form fields (name=value) before the file part.
- Use libraries like Apache HttpClient or OkHttp for cleaner APIs.
- Stream large files instead of buffering them in memory.

Scenario 53: Getting local machine hostname and IP

For logging, debugging, or showing system info, you often need the machine's hostname and IP.

Answer:

• Use InetAddress.getLocalHost() to retrieve the current machine's network identity.
• Call getHostName() for the hostname and getHostAddress() for the IP string.

Code:

```
import java.net.InetAddress;

public class LocalIPExample {
    public static void main(String[] args) {
        try {
            InetAddress local = InetAddress.getLocalHost();
            System.out.println("Host Name: " + local.getHostName());
            System.out.println("IP Address: " + local.getHostAddress());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• getLocalHost() resolves to the primary network interface address.
• getHostName() may perform a reverse DNS lookup, so it could be slow.
• Useful for diagnostics, but machines with multiple interfaces may need NetworkInterface API for detailed IPs.

Future scope:

• Use NetworkInterface.getNetworkInterfaces() to list all IPs (useful on servers with multiple NICs).
• Handle IPv6 addresses if the environment supports them.
• Combine with environment variables (like HOSTNAME) for fallback in containerized systems.

Scenario 54: Connecting to a test server with a self-signed SSL certificate

By default, Java's SSL implementation rejects unknown or self-signed certificates, since they're not signed by a trusted CA. For development, you sometimes want to bypass this restriction — but only in test environments.

Answer:

• Create a custom TrustManager that accepts all certs without validation.
• Initialize an SSLContext with this trust manager and apply it to HttpsURLConnection.
• Also override the HostnameVerifier to ignore hostname mismatches.
• This disables SSL validation — fine for development, but insecure for production.

Code:

```
import javax.net.ssl.*;
import java.net.URL;
import java.security.cert.X509Certificate;

public class SelfSignedSSLExample {
    public static void main(String[] args) {
        try {
            // Trust manager that accepts any certificate
            TrustManager[] trustAllCerts = new TrustManager[]{
                new X509TrustManager() {
                    public X509Certificate[] getAcceptedIssuers() { return null; }
                    public void checkClientTrusted(X509Certificate[] certs, String
authType) {}
                    public void checkServerTrusted(X509Certificate[] certs, String
authType) {}
                }
            };

            // Initialize SSL context
            SSLContext sc = SSLContext.getInstance("TLS");
            sc.init(null, trustAllCerts, new java.security.SecureRandom());
            HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());

            // Disable hostname verification
            HttpsURLConnection.setDefaultHostnameVerifier((hostname, session) ->
true);

            // Test request
            URL url = new URL("https://selfsigned.example.com");
            HttpsURLConnection conn = (HttpsURLConnection) url.openConnection();
            System.out.println("Response Code: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
```

```
    }
  }
}
```

Reasoning to interviewer:

- By overriding X509TrustManager, I tell Java to trust all server certs.
- Disabling hostname verification prevents errors when the cert CN doesn't match the URL.
- Useful for local dev with self-signed certs, but a serious risk in production because it opens the door to MITM attacks.

Future scope:

- For production, import the self-signed cert into a custom truststore and use that instead of trusting all.
- Consider Let's Encrypt or a corporate CA for valid certs.
- Use programmatic SSLContext to manage trust on a per-client basis, not globally.

Scenario 55: Throttling HTTP requests with rate limiting

Servers often impose request limits (like 5 requests/sec). Sending too fast causes throttling errors (429 Too Many Requests). The client must self-regulate.

Answer:

• Calculate the delay needed between requests (1000 / maxRequestsPerSecond).
• Use Thread.sleep() in simple cases to pause between calls.
• For more advanced control, use ScheduledExecutorService or token bucket algorithms.

Code (Simple sleep-based):

```
import java.net.HttpURLConnection;
import java.net.URL;

public class RateLimitingExample {
    public static void main(String[] args) {
        int maxRequestsPerSecond = 5;
        long delay = 1000 / maxRequestsPerSecond;

        for (int i = 0; i < 10; i++) {
            try {
                URL url = new URL("http://api.example.com/data");
                HttpURLConnection conn = (HttpURLConnection) url.openConnection();
                conn.setRequestMethod("GET");
                System.out.println("Response: " + conn.getResponseCode());

                Thread.sleep(delay); // enforce rate limit
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}
```

Code (Executor-based, cleaner):

```
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

public class ScheduledRateLimit {
    public static void main(String[] args) {
        int maxRequestsPerSecond = 5;
```

```
        ScheduledExecutorService scheduler =
Executors.newScheduledThreadPool(1);

        Runnable task = () -> {
            try {
                URL url = new URL("http://api.example.com/data");
                HttpURLConnection conn = (HttpURLConnection) url.openConnection();
                conn.setRequestMethod("GET");
                System.out.println("Response: " + conn.getResponseCode());
            } catch (Exception e) {
                e.printStackTrace();
            }
        };

        // schedule at fixed rate
        scheduler.scheduleAtFixedRate(task, 0, 200, TimeUnit.MILLISECONDS); //
5/sec
    }
}
```

Reasoning to interviewer:

- Rate limiting prevents hitting server-side quotas and avoids 429 or bans.
- Sleep-based throttling is simple but blocks threads; executor-based scheduling is cleaner and scalable.
- Real systems often use token bucket/leaky bucket algorithms for flexible burst handling.

Future scope:

- Backoff retries if server still responds with 429.
- Implement per-user or per-API-key throttling in multi-tenant apps.
- Integrate metrics so developers know if clients are close to limits.

Scenario 56: Running multiple HTTP requests asynchronously

If your app fetches from multiple APIs (say user profiles, orders, and notifications), running them one by one wastes time. You can dispatch all requests in parallel and wait for results.

Answer:

• Use a thread pool (ExecutorService) to submit multiple tasks, each making a request.
• Each task opens an HttpURLConnection, sends the request, and processes the response.
• This way, requests happen concurrently, reducing total latency.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class AsyncHttpExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(3);

        String[] urls = {
            "http://api.example.com/u1",
            "http://api.example.com/u2",
            "http://api.example.com/u3"
        };

        for (String u : urls) {
            executor.submit(() -> {
                try {
                    HttpURLConnection conn = (HttpURLConnection) new
URL(u).openConnection();
                    conn.setRequestMethod("GET");
                    System.out.println("Response from " + u + ": " +
conn.getResponseCode());
                } catch (Exception e) {
                    e.printStackTrace();
                }
            });
        }

        executor.shutdown();
    }
}
```

Reasoning to interviewer:

• ExecutorService provides a fixed pool of reusable threads, avoiding overhead of creating new ones.

• This pattern works well when APIs are independent, and results can be processed after all complete.

• Compared to sequential calls, latency is reduced to “slowest API time” rather than the sum of all.

Future scope:

• Use Future or CompletableFuture to gather results and compose them.

• Switch to Java 11 HttpClient with async methods (sendAsync()) for cleaner non-blocking APIs.

• Add timeouts, retries, and error aggregation.

Scenario 57: Connecting through a proxy with authentication

In corporate environments, outbound traffic often goes through a proxy requiring a username/password. Without configuring it, requests will fail with 407 Proxy Authentication Required.

Answer:

• Use Proxy class to specify the proxy host/port.
• Set a default Authenticator that provides proxy credentials.
• When opening a URL connection, Java will automatically send the credentials.

Code:

```
import java.net.*;

public class ProxyAuthExample {
    public static void main(String[] args) {
        // Set default authenticator for proxy credentials
        Authenticator.setDefault(new Authenticator() {
            @Override
            protected PasswordAuthentication getPasswordAuthentication() {
                return new PasswordAuthentication("user", "password".toCharArray());
            }
        });

        try {
            Proxy proxy = new Proxy(Proxy.Type.HTTP, new
InetSocketAddress("proxy.example.com", 8080));
            URL url = new URL("http://example.com");
            HttpURLConnection conn = (HttpURLConnection)
url.openConnection(proxy);
            System.out.println("Response: " + conn.getResponseCode());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Authenticator is a global hook Java uses whenever credentials are needed (both proxy and server).

• This setup works transparently with all HttpURLConnection calls once configured.

• The Proxy object allows connecting via a specific proxy host instead of the system default.

Future scope:

• Use HTTPS tunneling proxies (CONNECT method) if required.
• Externalize proxy config (host, port, creds) into environment variables or config files.
• For fine-grained control, implement a custom Authenticator that checks getRequestorType() and only supplies creds for proxies.

Scenario 58: Broadcasting chat messages on LAN

In a LAN chat app, a message from one client should reach all others without setting up direct connections to each peer. UDP broadcast solves this: you send a datagram to the broadcast address, and every client listening on that port gets it.

Answer:

• Use a DatagramSocket with broadcast mode enabled.
• Send packets to the broadcast address (255.255.255.255) or subnet broadcast (192.168.1.255).
• All clients that bound to the same port will receive the message.

Code:

```
import java.net.*;

public class BroadcastExample {
    public static void main(String[] args) {
        try (DatagramSocket socket = new DatagramSocket()) {
            socket.setBroadcast(true);

            String message = "Hello, everyone!";
            byte[] buffer = message.getBytes();

            DatagramPacket packet = new DatagramPacket(
                    buffer,
                    buffer.length,
                    InetAddress.getByName("255.255.255.255"),
                    8888
            );

            socket.send(packet);
            System.out.println("Broadcast message sent.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Broadcasting avoids maintaining N connections for N peers — one packet reaches all.

• Works best for local subnet communication. Routers often block broadcast

traffic beyond a subnet.

• Since UDP is unreliable, missed messages are possible.

Future scope:

• Use acknowledgments if delivery matters.

• Consider multicast groups for more structured subscriptions.

• Encrypt messages if the LAN isn’t trusted.

Scenario 59: Multicasting stock updates

For apps like stock tickers, you don't want to send N copies of the same update to N clients. Instead, use multicast: a single packet is delivered to all subscribed clients.

Answer:

• Use a MulticastSocket on the client to join a multicast group (230.0.0.0–239.255.255.255).
• The server sends packets to the group address.
• All clients in the group receive the updates.

Code (Client):

```
import java.net.*;

public class MulticastExample {
    public static void main(String[] args) {
        try (MulticastSocket socket = new MulticastSocket(4446)) {
            InetAddress group = InetAddress.getByName("230.0.0.0");
            socket.joinGroup(group);

            byte[] buffer = new byte[256];
            DatagramPacket packet = new DatagramPacket(buffer, buffer.length);

            socket.receive(packet);
            System.out.println("Received: " + new String(packet.getData()).trim());

            socket.leaveGroup(group);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Multicast is more efficient than broadcast — traffic goes only to group members.
• Useful for feeds like stock prices, sports scores, or live sensor data.
• Requires routers/switches that support multicast; not always enabled by default.

Future scope:

• Add authentication so only authorized clients join.

• Use TTL settings to control how far multicast traffic travels.

• Consider switching to WebSockets if crossing the public internet.

Scenario 60: Heartbeat messages in persistent TCP connections

When maintaining a long-lived TCP connection (like chat, trading, or IoT), you need to know if the client is still alive. Heartbeat (or keep-alive) messages confirm connectivity.

Answer:

• Use a background thread to periodically send a small marker (e.g., "PING\n") over the socket.
• The server can respond with "PONG" or just reset a timeout counter.
• This avoids hanging on dead connections.

Code (Client):

```
import java.io.OutputStream;
import java.net.Socket;

public class HeartbeatClient {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 9090)) {
            OutputStream out = socket.getOutputStream();

            while (true) {
                out.write("PING\n".getBytes());
                out.flush();
                System.out.println("Sent heartbeat...");
                Thread.sleep(5000); // every 5 seconds
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• TCP keep-alive exists at the OS level, but it's often too infrequent for real-time apps.
• Application-level heartbeats let you detect failures quickly and take action (reconnect, alert).
• Useful for detecting idle connections that were killed by firewalls/NAT.

Future scope:

• Add server-side timeout logic: drop client if no heartbeat for X seconds.

• Use ScheduledExecutorService instead of Thread.sleep() for more precise scheduling.

• Combine with exponential backoff reconnects when the connection is lost.

Scenario 61: Allowing quick server restarts with port reuse

Sometimes when restarting a TCP server quickly, the OS keeps the socket in TIME_WAIT, making the port appear “in use.” Enabling socket reuse solves this for development and testing.

Answer:

• Use an unbound ServerSocket or ServerSocketChannel.
• Call setReuseAddress(true) before binding.
• This allows the socket to rebind even if the OS is holding the port.

Code:

```
import java.net.*;

public class ReusePortServer {
    public static void main(String[] args) {
        try {
            ServerSocket server = new ServerSocket();
            server.setReuseAddress(true);
            server.bind(new InetSocketAddress(8080));
            System.out.println("Server started on port 8080 with reuse enabled.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Without setReuseAddress(true), the OS enforces a cooldown (TIME_WAIT).
• With it enabled, you can quickly restart a dev server without waiting.
• Useful in development, but in production you should weigh the risk of reusing sockets too aggressively.

Future scope:

• Combine with try-with-resources to ensure sockets close properly.
• Use NIO’s ServerSocketChannel for non-blocking reuse configurations.

Scenario 62: Streaming API responses instead of full load

Large JSON responses can be memory-heavy. Instead of waiting for the whole payload, you can stream it line by line.

Answer:

• Wrap the InputStream in a BufferedReader.
• Process each line as soon as it arrives.
• This reduces memory usage and latency for the first processed elements.

Code:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

public class StreamingResponseExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://api.example.com/largejson");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();

            try (BufferedReader reader = new BufferedReader(
                    new InputStreamReader(conn.getInputStream()))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    System.out.println("Received chunk: " + line);
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Streaming prevents OutOfMemoryError for very large responses.
• Allows processing results progressively (e.g., updating UI with first items).
• Works well when the API delivers line-delimited JSON or NDJSON.

Future scope:

• Integrate with JSON streaming parsers (Jackson Streaming API, Gson.stream()).

• Add backpressure if consumer processing is slower than network speed.

Scenario 63: Caching HTTP responses with validators

Fetching the same resource repeatedly is wasteful. HTTP caching uses validators (ETag, Last-Modified) to ask servers if the resource has changed.

Answer:

• Store the ETag or Last-Modified header from the first response.
• On subsequent requests, send If-None-Match or If-Modified-Since.
• If the server returns 304 Not Modified, use the cached version.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class CacheExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://example.com/resource");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();

            // Example: Send cached ETag
            conn.setRequestProperty("If-None-Match", "\"abc123etag\"");

            int code = conn.getResponseCode();
            if (code == HttpURLConnection.HTTP_NOT_MODIFIED) {
                System.out.println("Using cached response.");
            } else {
                System.out.println("Fetched new data.");
                // Save new ETag for next request
                String newEtag = conn.getHeaderField("ETag");
                System.out.println("New ETag: " + newEtag);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• If-None-Match with ETag is the strongest validator (byte-level changes).
• If-Modified-Since with Last-Modified works too, but timestamps can miss fast changes.
• This mechanism reduces bandwidth and improves response times.

Future scope:

- Implement a local cache (disk/DB) to persist responses.
- Handle expiry headers (Cache-Control, Expires).
- Use a caching HTTP client library (OkHttp, Apache HttpClient) for built-in support.

Scenario 64: Setting timeouts for slow API servers

If an API is slow, you don’t want your client to hang forever waiting. Timeouts ensure the app fails fast and can retry or notify the user.

Answer:

• Use setConnectTimeout(ms) to limit how long the client waits to establish a TCP connection.
• Use setReadTimeout(ms) to cap how long the client waits for data after connecting.
• Both values should be chosen based on SLA and network conditions.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class TimeoutURLConnectionExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://slow.example.com/api");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();

            conn.setConnectTimeout(5000); // 5 sec connect timeout
            conn.setReadTimeout(3000);    // 3 sec read timeout

            System.out.println("Response: " + conn.getResponseCode());
        } catch (Exception e) {
            System.out.println("Timeout or error: " + e.getMessage());
        }
    }
}
```

Reasoning to interviewer:

• Connection timeout protects against cases where the server host is unreachable.
• Read timeout prevents hanging when the server stalls mid-response.
• Setting both ensures the client remains responsive.

Future scope:

• Use retry with exponential backoff after a timeout.
• Switch to HttpClient (Java 11+) which has cleaner timeout APIs.
• Log timeout events for monitoring SLA breaches.

Scenario 65: Handling HTTP redirects manually

Sometimes you don’t want to auto-follow redirects — maybe you need to validate where it goes (security, logging, or analytics).

Answer:

• Call setInstanceFollowRedirects(false) to disable automatic redirect following.
• After getting the response, check if the status code is in the 3xx range.
• Fetch the Location header and decide whether to follow.

Code:

```
import java.net.HttpURLConnection;
import java.net.URL;

public class ManualRedirectExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://example.com/redirect");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setInstanceFollowRedirects(false);

            int code = conn.getResponseCode();
            if (code / 100 == 3) {
                String newUrl = conn.getHeaderField("Location");
                System.out.println("Redirected to: " + newUrl);
                // Optional: validate and then open a new connection
            } else {
                System.out.println("Response Code: " + code);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Manual handling lets you validate that the redirect target is safe.
• Useful in banking, payment gateways, or SSO flows where blindly following redirects could be risky.
• Provides visibility into how servers are redirecting traffic.

Future scope:

• Add whitelist/blacklist checks for allowed redirect domains.

• Limit redirect chains (avoid infinite loops).

• Log redirect targets for audit/debugging.

Scenario 66: Async HTTP calls with Java 11 HttpClient

Question: How can you make non-blocking HTTP requests in Java 11+?

Answer:

• Use the new HttpClient (Java 11).
• Call sendAsync(), which returns a CompletableFuture.
• Chain callbacks (thenApply, thenAccept) for async processing.

Code:

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.concurrent.CompletableFuture;

public class AsyncHttpClientExample {
    public static void main(String[] args) {
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("https://jsonplaceholder.typicode.com/posts/1"))
                .build();

        CompletableFuture<HttpResponse<String>> future =
                client.sendAsync(request, HttpResponse.BodyHandlers.ofString());

        future.thenApply(HttpResponse::body)
              .thenAccept(System.out::println)
              .join(); // wait for completion
    }
}
```

Reasoning to interviewer:

• HttpClient is the modern replacement for HttpURLConnection.
• Async avoids blocking threads — perfect for apps making many API calls.
• Futures allow composing multiple dependent calls.

Future scope:

• Use Executor for custom thread pools.
• Combine with CompletableFuture.allOf() for parallel API calls.
• Switch to WebSocket API in HttpClient for bidirectional streaming.

Scenario 67: Restricting SSL/TLS protocols and ciphers

Question: How do you force a Java client to use TLS 1.2 only?

Answer:

• Get an SSLContext for TLS.
• Customize SSLParameters to allow only TLSv1.2.
• Apply it to the connection before making requests.

Code:

```
import javax.net.ssl.*;
import java.net.URL;

public class TLS12Example {
    public static void main(String[] args) throws Exception {
        SSLContext sc = SSLContext.getInstance("TLS");
        sc.init(null, null, new java.security.SecureRandom());

        HttpsURLConnection conn = (HttpsURLConnection) new
URL("https://secure.example.com").openConnection();
        SSLSocketFactory factory = sc.getSocketFactory();
        conn.setSSLSocketFactory(factory);

        // Force TLS 1.2
        conn.setHostnameVerifier((h, s) -> true);
        System.out.println("Response: " + conn.getResponseCode());
    }
}
```

Reasoning to interviewer:

• Some servers reject TLS 1.0/1.1 for compliance (PCI-DSS, banking).
• Java allows restricting versions and ciphers to enforce security.
• You avoid downgrade attacks by pinning TLS 1.2/1.3.

Future scope:

• Fine-tune cipher suites using SSLParameters.setCipherSuites().
• Use strong truststores to avoid weak certs.
• Enable TLS 1.3 in modern JVMs.

Scenario 68: Listing all network interfaces and IPs

Question: How do you get all IP addresses and MAC addresses of your machine in Java?

Answer:

- Use NetworkInterface.getNetworkInterfaces().
- For each interface, list all InetAddress entries.
- Optionally, fetch hardware (MAC) address.

Code:

```
import java.net.*;
import java.util.*;

public class NetworkInterfacesExample {
    public static void main(String[] args) {
        try {
            Enumeration<NetworkInterface> nets =
NetworkInterface.getNetworkInterfaces();
            while (nets.hasMoreElements()) {
                NetworkInterface netint = nets.nextElement();
                System.out.println("Interface: " + netint.getName());

                byte[] mac = netint.getHardwareAddress();
                if (mac != null) {
                    System.out.print("MAC: ");
                    for (byte b : mac) {
                        System.out.format("%02X:", b);
                    }
                    System.out.println();
                }

                Enumeration<InetAddress> addrs = netint.getInetAddresses();
                while (addrs.hasMoreElements()) {
                    System.out.println("  IP: " + addrs.nextElement().getHostAddress());
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- This is useful for diagnostics, logging, or binding servers to specific interfaces.
- You can filter out loopback (127.0.0.1) and virtual interfaces.
- MAC addresses uniquely identify hardware (unless spoofed).

Future scope:

- Detect IPv6 vs IPv4 addresses.
- Monitor interface state (up/down).
- Use it in distributed systems for node identification.

Scenario 69: Designing a robust custom binary protocol (length-prefix + checksum)

Question: How do you frame/parse custom TCP messages safely so reads don't split/merge messages?

Answer:

• Prefix every message with a fixed-width length (e.g., 4-byte int), then payload, then an optional checksum (CRC32).
• On send: serialize → compute CRC → write [len][payload][crc].
• On receive: read exactly 4 bytes for length, then read that many payload bytes, then read checksum and verify.

Code:

```
import java.io.*;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.nio.ByteBuffer;
import java.util.zip.CRC32;

public class CustomProtocolClient {
    static byte[] frame(byte[] payload) throws IOException {
        CRC32 crc = new CRC32();
        crc.update(payload);
        long checksum = crc.getValue();

        ByteBuffer buf = ByteBuffer.allocate(4 + payload.length + 8);
        buf.putInt(payload.length);
        buf.put(payload);
        buf.putLong(checksum);
        return buf.array();
    }

    static byte[] readFully(InputStream in, int n) throws IOException {
        byte[] b = new byte[n];
        int off = 0;
        while (off < n) {
            int r = in.read(b, off, n - off);
            if (r == -1) throw new EOFException("stream closed mid-frame");
            off += r;
        }
        return b;
    }

    public static void main(String[] args) throws Exception {
        try (Socket s = new Socket()) {
            s.connect(new InetSocketAddress("127.0.0.1", 5555), 3000);
            s.setSoTimeout(4000);
```

```
            OutputStream out = new BufferedOutputStream(s.getOutputStream());
            InputStream in = new BufferedInputStream(s.getInputStream());

            byte[] payload = "HELLO_SERVER".getBytes("UTF-8");
            out.write(frame(payload));
            out.flush();

            // Read response
            int len = ByteBuffer.wrap(readFully(in, 4)).getInt();
            byte[] body = readFully(in, len);
            long crcSent = ByteBuffer.wrap(readFully(in, 8)).getLong();

            CRC32 crc = new CRC32(); crc.update(body);
            if (crc.getValue() != crcSent) throw new IOException("bad checksum");
            System.out.println("Server says: " + new String(body, "UTF-8"));
        }
    }
}
```

Reasoning to interviewer:

- TCP is a byte stream—messages can be fragmented/coalesced. Length-prefixing removes ambiguity.
- Checksums detect corruption and accidental framing bugs.
- Fixed-width integers + readFully avoid short-read pitfalls.

Future scope:

- Add version/opcode headers for extensibility.
- Use varint length to save bytes for small messages.
- Switch to NIO with a state machine for high concurrency servers.

Scenario 70: Tuning TCP socket options (latency, liveness, buffers)

Question: How do you reduce small-message latency and detect dead peers on a TCP connection?

Answer:

• Disable Nagle with setTcpNoDelay(true) to avoid bundling small writes (lower latency).
• Enable keep-alive with setKeepAlive(true) to let the OS probe dead connections.
• Tune receive/send buffers (setReceiveBufferSize, setSendBufferSize) for high-throughput flows.

Code:

```
import java.net.InetSocketAddress;
import java.net.Socket;

public class SocketTuningExample {
    public static void main(String[] args) throws Exception {
        try (Socket socket = new Socket()) {
            socket.setTcpNoDelay(true);          // low-latency small writes
            socket.setKeepAlive(true);           // detect half-open connections
            socket.setReceiveBufferSize(256 * 1024);
            socket.setSendBufferSize(256 * 1024);

            socket.connect(new InetSocketAddress("example.com", 9000), 3000);
            System.out.println("Connected with tuned options.");
            // ... use streams as usual
        }
    }
}
```

Reasoning to interviewer:

• TCP_NODELAY helps chat/command protocols where every millisecond matters.
• OS keep-alive intervals are long by default; for app-level liveness, combine with heartbeats/timeouts.
• Buffer sizes matter for high BDP links (high bandwidth × latency).

Future scope:

• Expose options via config so ops can tune per environment.
• For servers, also set setReuseAddress(true) before bind and backlog size.
• Consider SO_LINGER only when you understand shutdown semantics.

Scenario 71: WebSocket client using Java 11 HttpClient

Question: How do you consume a real-time feed (chat/ticker) with Java's built-in WebSocket API?

Answer:

• Use java.net.http.HttpClient's newWebSocketBuilder() to connect.
• Implement a WebSocket.Listener to handle onOpen, onText, onError, onClose.
• Send messages via webSocket.sendText(...) and manage backpressure with returned CompletableFuture.

Code:

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.WebSocket;
import java.util.concurrent.CompletionStage;

public class J11WebSocketClient {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        WebSocket ws = client.newWebSocketBuilder()
            .buildAsync(URI.create("wss://echo.websocket.events"), new
WebSocket.Listener() {
                @Override
                public void onOpen(WebSocket webSocket) {
                    System.out.println("Connected");
                    webSocket.sendText("Hello WS!", true);
                    WebSocket.Listener.super.onOpen(webSocket);
                }

                @Override
                public CompletionStage<?> onText(WebSocket webSocket,
CharSequence data, boolean last) {
                    System.out.println("Received: " + data);
                    return WebSocket.Listener.super.onText(webSocket, data, last);
                }

                @Override
                public void onError(WebSocket webSocket, Throwable error) {
                    System.err.println("Error: " + error.getMessage());
                    WebSocket.Listener.super.onError(webSocket, error);
                }

                @Override
                public CompletionStage<?> onClose(WebSocket webSocket, int
statusCode, String reason) {
                    System.out.println("Closed: " + statusCode + " " + reason);
```

```
                return WebSocket.Listener.super.onClose(webSocket, statusCode,
reason);
            }
        }).join();

        // keep alive briefly for demo; in real apps, keep running or use
CountDownLatch
        Thread.sleep(2000);
        ws.sendClose(WebSocket.NORMAL_CLOSURE, "bye");
    }
}
```

Reasoning to interviewer:

- Java 11’s WebSocket API integrates with HttpClient and CompletableFuture, making async flows natural.
- Listener methods are non-blocking; backpressure is handled via the returned futures.
- Works with both ws:// and wss:// (TLS).

Future scope:

- Reconnect with exponential backoff on close/error.
- Add ping/pong and application heartbeats.
- Use a message bus to fan-out events to the rest of the app.

Scenario 72: Making HTTP/2 multiplexed requests in Java 11

Question: How can you take advantage of HTTP/2 multiplexing (multiple requests over one TCP connection) in Java?

Answer:

• Java 11's HttpClient supports HTTP/2 by default if the server supports it.
• You can enable it explicitly with .version(HttpClient.Version.HTTP_2).
• Multiple concurrent sendAsync() calls reuse the same TCP connection.

Code:

```
import java.net.URI;
import java.net.http.*;

public class Http2Example {
    public static void main(String[] args) {
        HttpClient client = HttpClient.newBuilder()
                .version(HttpClient.Version.HTTP_2)
                .build();

        HttpRequest req1 =
HttpRequest.newBuilder(URI.create("https://http2.golang.org/reqinfo")).build();
        HttpRequest req2 =
HttpRequest.newBuilder(URI.create("https://http2.golang.org/reqinfo")).build();

        client.sendAsync(req1, HttpResponse.BodyHandlers.ofString())
              .thenAccept(r -> System.out.println("Resp1: " + r.statusCode()));

        client.sendAsync(req2, HttpResponse.BodyHandlers.ofString())
              .thenAccept(r -> System.out.println("Resp2: " + r.statusCode()));
    }
}
```

Reasoning to interviewer:

- HTTP/2 reduces connection overhead vs. HTTP/1.1 by multiplexing.
- In Java, you don't need special libraries—HttpClient handles it.
- Ideal for APIs with many small calls.

Future scope:

• Enable server push support if needed.

• Combine with TLS 1.3 for performance.

• Profile head-of-line blocking issues and fall back to parallel HTTP/1.1 if needed.

Scenario 73: Calling a gRPC service from Java

Question: How do you consume a gRPC service in Java?

Answer:

• gRPC uses HTTP/2 under the hood with Protocol Buffers.
• Generate Java stubs from .proto files using protoc.
• Use ManagedChannel and a stub client to make calls.

Code (simplified client):

```
import io.grpc.ManagedChannel;
import io.grpc.ManagedChannelBuilder;

// Assume GreeterGrpc and HelloRequest generated from proto
public class GrpcClient {
    public static void main(String[] args) {
        ManagedChannel channel = ManagedChannelBuilder
                .forAddress("localhost", 50051)
                .usePlaintext()
                .build();

        GreeterGrpc.GreeterBlockingStub stub =
GreeterGrpc.newBlockingStub(channel);
        HelloRequest request = HelloRequest.newBuilder().setName("Rajat").build();
        HelloReply response = stub.sayHello(request);

        System.out.println("Server response: " + response.getMessage());
        channel.shutdown();
    }
}
```

Reasoning to interviewer:

- gRPC is common in microservices (low latency, type-safe).
- Protobuf ensures compact binary serialization.
- Java tooling auto-generates client/server stubs from proto definitions.

Future scope:

- Add TLS with useTransportSecurity().
- Use async stub for streaming.
- Combine with service discovery for distributed systems.

Scenario 74: Zero-copy file transfer with FileChannel

Question: How do you send large files efficiently from server to client in Java?

Answer:

• Use FileChannel.transferTo() (zero-copy).
• It lets the OS kernel transfer bytes directly from file to socket buffer without user-space copying.
• Much faster for GB-size files compared to manual read/write loops.

Code (server side):

```
import java.io.RandomAccessFile;
import java.net.InetSocketAddress;
import java.nio.channels.*;

public class ZeroCopyFileServer {
    public static void main(String[] args) throws Exception {
        try (ServerSocketChannel server = ServerSocketChannel.open()) {
            server.bind(new InetSocketAddress(9090));
            System.out.println("Waiting for client...");
            SocketChannel client = server.accept();

            try (RandomAccessFile file = new RandomAccessFile("bigfile.zip", "r");
                 FileChannel fileChannel = file.getChannel()) {

                long size = fileChannel.size();
                long transferred = fileChannel.transferTo(0, size, client);
                System.out.println("Transferred " + transferred + " bytes.");
            }
        }
    }
}
```

Reasoning to interviewer:

• transferTo() avoids copying file data into JVM heap → better CPU usage, less GC pressure.
• Used in high-performance servers (e.g., NIO frameworks like Netty).
• Ideal for video streaming, backups, file servers.

Future scope:

• Add throttling if you don’t want to saturate the link.

• Handle partial transfers with a loop (while(transferred < size)).

• Combine with SSL/TLS if confidentiality is required.

Scenario 75: Secure WebSocket with mutual TLS

Question: How do you connect a WebSocket client that must present a client certificate (mutual TLS)?

Answer:

• Load a client certificate into a KeyStore.
• Initialize an SSLContext with both trust managers (server certs) and key managers (client certs).
• Pass this SSL context into the WebSocket client.

Code:

```
import javax.net.ssl.*;
import java.io.FileInputStream;
import java.net.URI;
import java.security.KeyStore;
import java.net.http.*;

public class SecureWsClient {
    public static void main(String[] args) throws Exception {
        // Load client certificate
        KeyStore ks = KeyStore.getInstance("PKCS12");
        ks.load(new FileInputStream("client.p12"), "password".toCharArray());

        KeyManagerFactory kmf =
KeyManagerFactory.getInstance(KeyManagerFactory.getDefaultAlgorithm());
        kmf.init(ks, "password".toCharArray());

        SSLContext sslContext = SSLContext.getInstance("TLS");
        sslContext.init(kmf.getKeyManagers(), null, null);

        HttpClient client = HttpClient.newBuilder()
                .sslContext(sslContext)
                .build();

        client.newWebSocketBuilder()
                .buildAsync(URI.create("wss://secure.example.com/chat"), new
WebSocket.Listener() {})
                .join();

        System.out.println("Connected to secure WebSocket with mutual TLS.");
    }
}
```

Reasoning to interviewer:

• Standard WebSockets over wss:// use TLS, but mutual TLS requires the client to present its own certificate.

• Setting up KeyManagerFactory ensures the client cert is sent during handshake.

• Useful for enterprise/banking apps with strict identity checks.

Future scope:

• Add TrustManager to pin server certs.

• Rotate client certs automatically with keystore reloads.

• Support fallback for non-mTLS connections if needed.

Scenario 76: Non-blocking UDP with DatagramChannel

Question: How do you implement non-blocking UDP communication in Java NIO?

Answer:

- Use DatagramChannel instead of DatagramSocket.
- Configure channel as non-blocking and register with a Selector.
- Read/write packets when the channel is ready.

Code:

```
import java.net.InetSocketAddress;
import java.nio.ByteBuffer;
import java.nio.channels.DatagramChannel;
import java.nio.channels.Selector;
import java.nio.channels.SelectionKey;

public class NioUdpExample {
    public static void main(String[] args) throws Exception {
        Selector selector = Selector.open();
        DatagramChannel channel = DatagramChannel.open();
        channel.configureBlocking(false);
        channel.bind(new InetSocketAddress(9999));
        channel.register(selector, SelectionKey.OP_READ);

        System.out.println("Listening for UDP packets on port 9999...");
        ByteBuffer buffer = ByteBuffer.allocate(1024);

        while (true) {
            selector.select();
            for (SelectionKey key : selector.selectedKeys()) {
                if (key.isReadable()) {
                    buffer.clear();
                    InetSocketAddress addr = (InetSocketAddress)
channel.receive(buffer);
                    buffer.flip();
                    System.out.println("From " + addr + ": " +
                            new String(buffer.array(), 0, buffer.limit()));
                }
            }
            selector.selectedKeys().clear();
        }
    }
}
```

Reasoning to interviewer:

- Traditional DatagramSocket is blocking; with NIO you can scale to thousands of sockets.
- Selector lets one thread manage multiple channels.
- Perfect for high-performance real-time apps (games, streaming).

Future scope:

- Add OP_WRITE for sending messages.
- Use multiple selectors for CPU scaling.
- Encrypt packets with DTLS if security is needed.

Scenario 77: Simulating many clients for load testing

Question: How do you simulate thousands of concurrent socket clients in Java?

Answer:

• Use thread pools for TCP clients, or better, use NIO to scale without 1:1 threads.
• Each client opens a SocketChannel or Socket to the server, sends test messages, and reads responses.
• Useful for stress testing server capacity.

Code (simplified TCP load test):

```
import java.net.Socket;
import java.io.OutputStream;

public class LoadTestClients {
    public static void main(String[] args) throws Exception {
        int clientCount = 100; // simulate 100 clients

        for (int i = 0; i < clientCount; i++) {
            new Thread(() -> {
                try (Socket socket = new Socket("localhost", 9090);
                     OutputStream out = socket.getOutputStream()) {
                    out.write("Hello from client\n".getBytes());
                    out.flush();
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }).start();
        }

        System.out.println(clientCount + " clients launched.");
    }
}
```

Reasoning to interviewer:

- For a quick load test, threads + sockets are fine (hundreds of clients).
- For thousands/millions, use NIO or dedicated frameworks (Netty, Vert.x).
- Helps benchmark throughput, latency, and server stability.

Future scope:

• Capture metrics (RTT, failed connections).

• Add ramp-up logic (e.g., 100 clients per second).

• Integrate with tools like JMeter, Gatling for distributed load testing.

Scenario 78: Detecting and closing idle/half-open TCP connections (server-side)

Question: How can you detect clients that stopped sending data (idle/half-open) and close them to free resources?

Answer:

• Set a per-socket read timeout (setSoTimeout) so a blocked read() throws SocketTimeoutException after N seconds of inactivity.
• Keep reading in a loop; on timeout, log and close the connection.
• Optionally, send/expect application-level heartbeats (“PING/PONG”) for finer control, and enable OS keep-alive as a safety net.

Code:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.ServerSocket;
import java.net.Socket;

public class IdleCloseServer {
    public static void main(String[] args) {
        final int PORT = 9090;
        final int IDLE_MS = 15_000; // 15s without data -> drop

        try (ServerSocket server = new ServerSocket(PORT)) {
            System.out.println("Listening on " + PORT);
            while (true) {
                Socket client = server.accept();
                client.setSoTimeout(IDLE_MS);   // read timeout
                client.setKeepAlive(true);      // OS-level keepalive (coarse)

                new Thread(() -> handle(client, IDLE_MS)).start();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static void handle(Socket client, int idleMs) {
        String who = client.getRemoteSocketAddress().toString();
        System.out.println("Client connected: " + who);
        try (BufferedReader in = new BufferedReader(new
InputStreamReader(client.getInputStream()))) {
            String line;
            while (true) {
                try {
                    line = in.readLine();          // blocks until data or timeout
                    if (line == null) break;       // EOF -> client closed cleanly
```

```
                System.out.println("[" + who + "] " + line);
                // Optionally: update last-seen timestamp, respond, etc.
            } catch (java.net.SocketTimeoutException idle) {
                System.out.println("Idle timeout (" + idleMs + "ms) for " + who + " ->
closing");
                break;
            }
        }
    } catch (Exception e) {
        System.out.println("Error with " + who + ": " + e.getMessage());
    } finally {
        try { client.close(); } catch (Exception ignored) {}
    }
  }
}
```

Reasoning to interviewer:

- setSoTimeout turns indefinite reads into bounded waits; no thread gets stuck forever.
- Distinguishes clean closes (EOF) from silent drops (timeout).
- OS keep-alive is slow (minutes), so app-level timeouts/heartbeats provide timely detection.

Future scope:

- Switch to NIO Selector and track lastSeen timestamps for thousands of clients.
- Add heartbeats and drop clients that miss K consecutive beats.
- Expose the idle threshold via config and emit metrics/alerts for idle disconnects.

Scenario 79: Streaming file upload over HTTP using Java 11 HttpClient

Question: How do you upload a large file to an HTTP endpoint without buffering it all in memory?

Answer:

• Use Java 11's HttpClient and BodyPublishers.ofFile(Path) to stream the file.
• Set Content-Type appropriately (e.g., application/octet-stream or an API-specific type).
• The client streams directly from disk to the socket—no huge heap usage.

Code:

```
import java.net.URI;
import java.net.http.*;
import java.nio.file.Path;

public class StreamingUploadExample {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://files.example.com/upload"))
            .header("Content-Type", "application/octet-stream")
            .POST(HttpRequest.BodyPublishers.ofFile(Path.of("profile.jpg")))
            .build();

        HttpResponse<String> resp = client.send(request,
HttpResponse.BodyHandlers.ofString());
        System.out.println("Status: " + resp.statusCode());
        System.out.println("Body: " + resp.body());
    }
}
```

Reasoning to interviewer:

• ofFile streams bytes from disk; it's simpler and safer than manual HttpURLConnection writes.
• Works well with large payloads and plays nicely with HTTP/2.
• You can still add auth headers (e.g., Authorization) or query params as needed.

Future scope:

• For multipart uploads (fields + file), build a multipart/form-data body or use a library (OkHttp/Apache HC).

• Add retry/backoff and checksum verification.

• Implement progress reporting via a custom BodyPublisher wrapper.

Scenario 80: gRPC bidirectional streaming client in Java

Question: How do you implement a client that both sends and receives a stream of messages concurrently with gRPC?

Answer:

• Use the async stub (non-blocking) generated from your .proto.
• Create a StreamObserver<Req> from the stub call; keep a reference and call onNext() to send.
• Provide a StreamObserver<Resp> to handle server messages, onError, and onCompleted.

Code (client skeleton):

```
import io.grpc.ManagedChannel;
import io.grpc.ManagedChannelBuilder;
import io.grpc.stub.StreamObserver;

// Assume StockServiceGrpc, PriceUpdate, PriceRequest are generated from
stock.proto
public class GrpcBidiClient {
    public static void main(String[] args) throws Exception {
        ManagedChannel channel = ManagedChannelBuilder
            .forAddress("localhost", 50051)
            .usePlaintext()
            .build();

        StockServiceGrpc.StockServiceStub stub =
StockServiceGrpc.newStub(channel);

        StreamObserver<PriceRequest> clientStream =
            stub.streamPrices(new StreamObserver<PriceUpdate>() {
                @Override public void onNext(PriceUpdate update) {
                    System.out.println("Update: " + update.getSymbol() + " -> " +
update.getPrice());
                }
                @Override public void onError(Throwable t) {
                    System.err.println("Stream error: " + t.getMessage());
                }
                @Override public void onCompleted() {
                    System.out.println("Server completed");
                }
            });

        // Send subscription requests (client -> server)
        clientStream.onNext(PriceRequest.newBuilder().setSymbol("AAPL").build());
        clientStream.onNext(PriceRequest.newBuilder().setSymbol("GOOG").build());

        // ... later, finish sending
```

```
        clientStream.onCompleted();

        // Keep process alive briefly to receive updates (use latch in real app)
        Thread.sleep(3000);
        channel.shutdownNow();
    }
}
```

Reasoning to interviewer:

- Bidirectional streaming maps naturally to live feeds (prices, chat): both sides can push at any time.
- Async stubs + StreamObserver avoid blocking; you can wire them into reactive pipelines.
- Backpressure is handled by gRPC/HTTP2 flow control; still, design messages to be small and frequent.

Future scope:

- Add TLS/mTLS for secure channels.
- Use onReadyHandler to implement client-side backpressure.
- Reconnect with exponential backoff and resume subscriptions on failures.